

メッシュシェーダー入門

GPU 駆動ジオメトリを、図・数式・演習で基礎から理解する

NVIDIA / AMD Radeon Windows / Linux

Vulkan VK_EXT_mesh_shader / Direct3D 12



編集・執筆: OpenAI

第1版・2026年8月24日

はじめに

メッシュシェーダーは、従来の頂点シェーダー中心の描画パイプラインを、より計算シェーダーに近い協調実行モデルへ組み替える仕組みである。一言でいえば、GPU 上の複数スレッドが小さな三角形の集まりを共同で読み込み、不要なものを捨て、必要な頂点とプリミティブだけをラスタライザへ渡す。この「小さな三角形の集まり」が**メッシュレット**である。

ただし、メッシュシェーダーは魔法の高速化スイッチではない。入力アセンブラが行っていた頂点再利用の準備をアセット側へ移し、ワークグループの大きさ、出力上限、共有メモリ、カリング、LOD、同期を自分で設計する必要がある。その代わり、ジオメトリ処理を GPU 主導で組み立てられる自由を得る。

本書の目的は、API の関数名だけを暗記することではない。次の三つを、手を動かしながら身につけることである。

- 従来パイプラインと何が変わるのかを、初心者でも説明できる。
- NVIDIA と AMD Radeon のどちらでも成立する設計原則を理解する。
- Windows と Linux で、Vulkan を共通経路として実験し、Windows では Direct3D 12 との対応関係も読める。

要点

本書の主経路は、クロスベンダーかつ Windows/Linux で利用できる Vulkan の `VK_EXT_mesh_shader` である。Direct3D 12 は Windows 向けの対応章として扱う。GPU 名やドライバ版を固定して判定せず、必ず実行時に拡張・機能・上限を問い合わせる。

本書で作るもの

最終章では、次の機能を持つ「メッシュレット・ビューア」を設計する。

- 通常の頂点パイプラインとメッシュシェーダーパイプラインの切り替え
- メッシュレットごとの色分け
- 視錐台カリングと法線コーンカリング
- LOD 切り替え
- 可視メッシュレット数、出力頂点数、出力三角形数、GPU 時間の表示

- 機能非対応 GPU でのフォールバック

前提知識

C++ の基本文法、GPU で三角形を描く概念、Vulkan または Direct3D 12 の「Hello Triangle」を一度見た経験があると読みやすい。ただし、ベクトル、行列、平面、座標変換、ワークグループ、メモリアライメントは必要な場所で改めて解説する。

注意

本書の API 名・上限・ドライバ挙動は、2026 年 8 月 24 日時点の公開仕様と公式資料を基にしている。実装時は手元の SDK、検証レイヤ、GPU ドライバ、API 仕様を確認すること。特に「最大頂点数」「最大プリミティブ数」「ワークグループサイズ」はハードコードせず、実機のプロパティを問い合わせる。

本書の読み方

各章は、次のリズムで構成している。

1. **全体像**: 何のための仕組みかを図で把握する。
2. **背景知識**: 数学、GPU 実行モデル、データ構造を必要な分だけ学ぶ。
3. **コード**: Vulkan/GLSL または Direct3D 12/HLSL の断片を読む。
4. **演習**: 紙と鉛筆、または小さなコード変更で確認する。
5. **解説**: 付録 C で考え方を照合する。

最初から完全なレンダラを作ろうとすると、ウィンドウ、スワップチェーン、同期、パイプラインキャッシュなどの周辺作業に埋もれやすい。そこで本書のハンズオンは、Khronos の Vulkan Samples または Microsoft の DirectX Graphics Samples を土台にし、メッシュシェーダー固有部分へ集中する。公式サンプルは継続的に更新されるため、学習用の起点として適している [9, 13]。

目次

はじめに	i
本書の読み方	iii
第 I 部 まず全体像をつかむ	1
第 1 章 メッシュシェーダーとは何か	2
1.1 まずは工場にたとえる	2
1.2 Mesh Shader が出力するもの	3
1.3 何がうれしいのか	4
1.4 API と呼び名を対応させる	5
1.5 最小の心のモデル	5
第 2 章 GPU の実行モデルを理解する	7
2.1 CPU のスレッドと GPU のスレッドは何が違うか	7
2.2 識別子を読み分ける	7
2.3 メモリ階層	8
2.4 出力上限は設計条件である	9
2.5 分岐とダイバージェンス	9
第 3 章 メッシュレットというデータ単位	11
3.1 大きなメッシュを小さく切る	11
3.2 2 段階のインデックス	11
3.3 良い分割の条件	12
3.4 単純な貪欲分割	13
3.5 境界情報を添える	14

第 4 章	カリングと LOD に必要な数学	15
4.1	座標をベクトルとして扱う	15
4.2	内積は「向きの近さ」を測る	15
4.3	外積は面の法線を作る	16
4.4	同次座標と行列	16
4.5	境界球と視錐台平面	17
4.6	法線コーンによる背面カリング	18
4.7	画面上の誤差と LOD	18
第 II 部	動かして理解する	20
第 5 章	開発環境と機能検出	21
5.1	どの経路で学ぶか	21
5.2	必要な道具	21
5.3	公式 Vulkan Samples をビルドする	22
5.4	Vulkan で拡張を確認する	22
5.5	Direct3D 12 で Tier を確認する	25
5.6	フォールバックを最初から設計する	26
第 6 章	最初の三角形を出力する	27
6.1	GLSL の Mesh Shader	27
6.2	Vulkan の graphics pipeline	28
6.3	HLSL の Mesh Shader	30
6.4	同じ HLSL を Vulkan へ使う	31
6.5	画面が出ないときの順序	31
第 7 章	バッファからメッシュレットを描く	33
7.1	データレイアウトを固定する	33
7.2	GLSL で読み出す	33
7.3	HLSL 版の対応	35

7.4	CPU で最小データを作る	37
第 8 章	Task/Amplification Shader で可視仕事を圧縮する	39
8.1	なぜ前段があるのか	39
8.2	GLSL Task Shader の基本形	39
8.3	HLSL Amplification Shader	41
8.4	atomic compaction から subgroup compaction へ	43
8.5	法線コーンを追加する	43
8.6	Task/Amplification Shader を使わない方がよい場合	44
第 9 章	LOD と GPU 駆動描画	45
9.1	LOD は三角形数だけの問題ではない	45
9.2	画面誤差で選ぶ	45
9.3	3 つの実装段階	46
9.4	indirect dispatch	46
9.5	occlusion culling の注意	47
9.6	LOD の継ぎ目	47
第 III 部	実用データとレンダラへ育てる	48
第 10 章	アセット前処理パイプライン	49
10.1	ランタイムより先にデータを設計する	49
10.2	入力メッシュを清掃する	49
10.3	隣接情報を作る	50
10.4	メッシュレットを生成する	51
10.5	境界球を生成する	51
10.6	法線コーンを生成する	52
10.7	LOD と誤差を生成する	52
10.8	binary format を version 化する	53
10.9	統計と可視化	53

第 11 章	頂点属性・マテリアル・圧縮	55
11.1	位置だけから実用頂点へ	55
11.2	量子化位置を復元する	55
11.3	octahedral normal encoding	56
11.4	AoS と SoA	57
11.5	per-vertex と per-primitive	57
11.6	material を混ぜるか分けるか	58
11.7	透明・両面材質	58
第 12 章	アニメーションとプロシージャル生成	60
12.1	スキニングをどこで行うか	60
12.2	スキニングの式	60
12.3	静的 bounds は変形後を含むか	61
12.4	morph target	61
12.5	プロシージャル geometry	62
12.6	tessellation の置き換え	63
第 13 章	同期・メモリ・リソース寿命	64
13.1	正しいデータでも同期がなければ壊れる	64
13.2	Vulkan Synchronization2 の例	64
13.3	upload から Mesh Shader read へ	65
13.4	D3D12 の barrier	66
13.5	frame 間の競合を避ける	66
13.6	descriptor 更新と寿命	67
13.7	statistics counter の読み戻し	67
第 IV 部	性能・品質・製品化	69
第 14 章	NVIDIA と Radeon を横断して最適化する	70
14.1	最初に測定条件を固定する	70

14.2	ボトルネックを段階分解する	71
14.3	meshlet profile を sweep する	71
14.4	出力宣言を小さくする	72
14.5	workgroup size と subgroup	72
14.6	i 番 lane が i 番 output を書く	73
14.7	payload と共有メモリを小さくする	73
14.8	Task 段の採算を数える	74
14.9	raster と small triangle	74
14.10	メーカー固有最適化を隔離する	74
第 15 章	デバッグとプロファイリング	76
15.1	デバッグ層を常時使える構成にする	76
15.2	段階的な可視化 mode	76
15.3	CPU reference を持つ	77
15.4	canary と範囲検査	77
15.5	GPU capture で見る順序	78
15.6	timestamp query の設計	78
15.7	代表的な profiler	79
15.8	非決定性を制御する	79
第 16 章	既存レンダラから移行する	81
16.1	一度に置き換えない	81
16.2	共通 asset 表現	81
16.3	shader code を共有する	82
16.4	pipeline key と cache	83
16.5	テスト matrix	83
16.6	失敗時の安全な降格	84
16.7	品質比較	84
第 17 章	総合演習: メッシュレット・ビューアを作る	85

17.1	完成像	85
17.2	directory 構成	85
17.3	Milestone 0: baseline	86
17.4	Milestone 1: feature detection と固定 triangle	86
17.5	Milestone 2: converter と 1 meshlet	87
17.6	Milestone 3: 複数 meshlet と bounds	87
17.7	Milestone 4: culling	87
17.8	Milestone 5: LOD	88
17.9	Milestone 6: indirect と同期	88
17.10	Milestone 7: cross-vendor tuning	88
17.11	UI 設計	89
17.12	提出物と評価 rubric	90
17.13	ここまでで身に付いたこと	90
付録 A	数学・GPU 演算早見表	92
A.1	ベクトル	92
A.2	変換	92
A.3	カリング	93
A.4	並列 compaction	93
A.5	容量計算	94
付録 B	Vulkan と Direct3D 12 対応表	95
B.1	stage・command・intrinsic	95
B.2	Vulkan の機能構造	96
B.3	Direct3D 12 の必須規模	96
B.4	shader interface の確認事項	97
B.5	feature fallback 表	97
付録 C	演習問題の解説	98

付録 D 用語集	105
参考資料	108
参考文献	109
あとがき	111

目次

1.1	従来パイプラインとメッシュシェーダーパイプラインの概念差	2
1.2	1 ワークグループが小さなメッシュの出力を完結させる	3
2.1	ワークグループと subgroup の関係を表す概念図	7
3.1	連結した三角形群を複数メッシュレットへ分割する	11
3.2	メッシュレットの代表的なデータ構造	12
4.1	内積と 2 つの方向のなす角	15
4.2	平面と境界球の判定。法線が内側を向く規約の例	17
4.3	法線コーンの概念。実装前に符号と方向をテストする	18
5.1	実行時機能検出によるパイプライン選択	26
8.1	不可視候補を圧縮して、必要な Mesh Shader だけを発行する	39
9.1	クラスタ階層 LOD の概念。画面誤差が大きい node だけ子へ降りる	45
9.2	Compute pass で可視リストと indirect 引数を作る構成	46
10.1	オフラインアセット変換の推奨工程	49
11.1	AoS と SoA の概念。必要属性だけ連続読み出しできるかが違う	57
13.1	frame ごとに書き換え buffer 領域を分離する ring 構成	67
14.1	性能を pipeline 区間へ分けて考える	71
17.1	総合演習の構成	85

表目次

1.1	メッシュシェーダーが解決しやすい課題	4
1.2	Vulkan と Direct3D 12 のおおまかな対応	5
2.1	シェーダー内の代表的な識別子	8
2.2	GPU シェーダーから見た主な記憶領域	8
2.3	学習用に比較するメッシュレットプロファイル	9
3.1	メッシュレット分割の代表的な評価軸	13
4.1	座標系で明文化すべき項目	17
5.1	本書で扱う実装経路	21
7.1	学習用バッファ構成	33
9.1	GPU 駆動化の段階	46
10.1	メッシュアセット header に含めたい項目	53
11.1	代表的な属性形式とトレードオフ	55
11.2	マテリアル分割方針	58
12.1	スキニング位置の選択	60
14.1	推奨する profile sweep 例	71
14.2	shader variant を増やす判断	72
14.3	payload へ入れる判断	73
15.1	不具合の症状から最初に見る場所	76
15.2	代表的な解析 tool と使い分け	79

16.1	最低限の cross-platform test matrix	83
17.1	表示する asset 統計	87
17.2	ビューアに必要な操作	89
17.3	総合演習の自己評価 rubric	90
A.1	メッシュシェーダーで使うベクトル演算	92
A.2	代表的なカリング判定	93
B.1	Mesh Shader API 対応表	95
B.2	Vulkan で確認する主な構造と field	96
B.3	機能組み合わせごとの経路	97
D.1	メッシュシェーダー関連用語	105

第Ⅰ部

まず全体像をつかむ

第 1

メッシュシェーダーとは何か

要点

メッシュシェーダーは、GPU 上の小さな協調グループが、これから描く頂点とプリミティブをまとめて生成するシェーダーステージである。従来の「入力された頂点を 1 個ずつ変換する」考え方から、「小さなメッシュの塊を 1 グループで処理する」考え方へ移る。

1.1 まずは工場にたとえる

従来の描画パイプラインを、部品がベルトコンベアを流れる工場にたとえよう。頂点シェーダーは、流れてきた頂点を 1 個ずつ加工する担当者である。三角形の組み立て方は、入力アセンブラが持つインデックス列でほぼ決まっている。途中にテッセレーションやジオメトリシェーダーを追加できるが、工程が細かく分かれ、前後の受け渡し形式も固定されやすい。

メッシュシェーダーでは、32 人や 64 人などの作業班が小箱を受け取る。箱には「このメッシュレットを処理せよ」という情報が入っている。班員は協力して頂点を読み、不要なら箱ごと捨て、必要なら頂点配列と三角形配列をその場で出力する。この小箱が後述する**メッシュレット**であり、作業班が**ワークグループ**である。

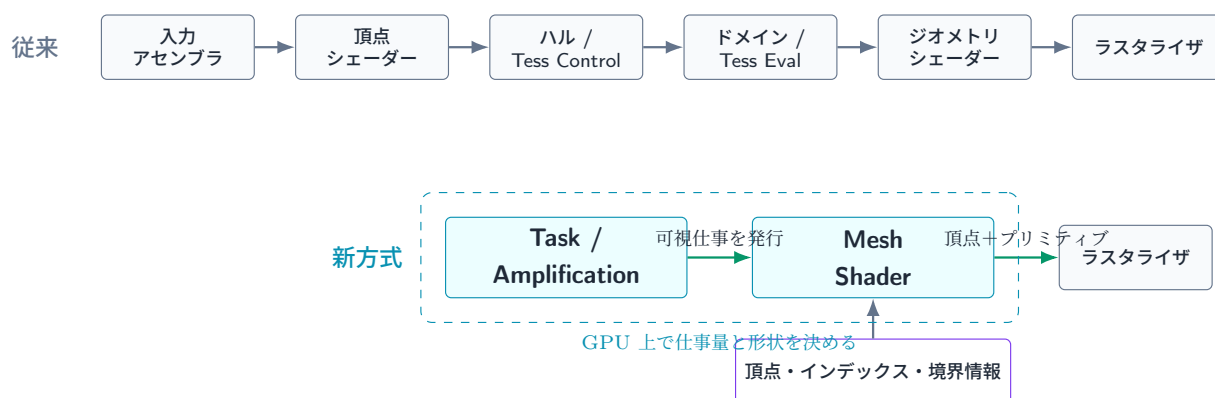


図 1.1: 従来パイプラインとメッシュシェーダーパイプラインの概念差

Direct3D 12 では、**Amplification Shader** と **Mesh Shader** が従来の頂点・テッセレーション・ジオメトリ系ステージを置き換える。Vulkan の `VK_EXT_mesh_shader` では、ほぼ同じ役割を **Task Shader** と **Mesh Shader** と呼ぶ。Task/Amplification 段は任意であり、最初は Mesh Shader だけで三角形を出してよい [1, 4]。

1.2 Mesh Shader が出力するもの

Mesh Shader の仕事は、最終的にラスライザへ次を渡すことである。

1. **頂点数**: 今回のワークグループが出力する頂点の個数。
2. **プリミティブ数**: 三角形、線、点の個数。
3. **頂点属性**: クリップ座標、法線、UV、色など。
4. **接続情報**: どの出力頂点を組み合わせて三角形にするか。
5. **プリミティブ属性**: マテリアル ID、カリング制御など、三角形単位の情報。

重要なのは、出力数が「ディスパッチ時点の固定値」ではなく、ワークグループごとに決められる点である。見えないメッシュレットなら 0 頂点・0 プリミティブを出力し、見えるなら必要な量だけを出力できる。ただし、シェーダー宣言とデバイスプロパティが定める最大値を超えてはならない。



図 1.2: 1 ワークグループが小さなメッシュの出力を完結させる

1.3 何がうれしいのか

表 1.1: メッシュシェーダーが解決しやすい課題

課題	従来の典型	メッシュシェーダーでの考え方
細粒度カリング	CPU や Compute Shader で可視リストを作り、別の描画へ渡す	Task/Mesh 段でメッシュレット単位に判定し、0 出力または仕事を圧縮
大量の小物体	Draw Call や状態切り替えが増える	大きなディスパッチから GPU が小さな仕事を展開
可変トポロジ	Geometry Shader や複数パスに頼る	ワークグループ内で頂点・プリミティブ数を決める
LOD	CPU が距離判定し、別メッシュを選ぶ	メッシュレット階層やクラスタ単位で GPU 選択
データ形式	固定的な頂点入力に寄せる	ByteAddress/Storage Buffer から任意圧縮形式を展開

一方で、メッシュシェーダーは「必ず高速になる魔法」ではない。単純な全画面三角形、頂点数が少ない UI、すでに頂点処理が軽く CPU 負荷も小さい場面では、移行コストに見合わないことがある。また、巨大な出力配列、重い Task Shader、非効率なメッシュレット分割は、従来方式より遅くなり得る。

直感でつかむ

メッシュシェーダーの中心価値は、シェーダー名ではなく「仕事の単位を、頂点から小さなメッシュへ変える」ことにある。その単位に境界球、法線コーン、LOD 誤差を持たせると、GPU が描画前に賢く捨てられる。

1.4 API と呼び名を対応させる

表 1.2: Vulkan と Direct3D 12 のおおまかな対応

概念	Vulkan	Direct3D 12 / HLSL
機能	VK_EXT_mesh_shader	Mesh Shader Tier 1
前段	Task Shader	Amplification Shader
本体	Mesh Shader	Mesh Shader
起動	vkCmdDrawMeshTasksEXT	DispatchMesh
出力数設定	SetMeshOutputsEXT	SetMeshOutputCounts
前段から本体を発行	EmitMeshTasksEXT	DispatchMesh (AS 内)
バイナリ	SPIR-V	DXIL
代表的言語	GLSL / HLSL	HLSL

本書では、OS をまたいで学べる Vulkan を主軸とし、Windows では Direct3D 12 の対応コードを併記する。HLSL を DXC で SPIR-V へコンパイルする構成も可能であるため、「シェーダー言語は HLSL に統一し、API だけ切り替える」という設計も選べる [7, 8]。

1.5 最小の心のモデル

1. CPU は「何個のワークグループを起動するか」を命令する。
2. 各グループは担当メッシュレットを決める。
3. 必要なら境界情報で可視性を判定する。
4. 可視なら頂点と局所三角形を読み、出力配列へ書く。
5. ラスタライザが三角形をピクセルへ変換し、Fragment/Pixel Shader が色を決める。

演習 1.1: 用語を自分の言葉にする

次の 3 語を、それぞれ 30 字から 60 字で説明せよ。[ワークグループ](#)、[メッシュレット](#)、[Task/Amplification Shader](#)。さらに、Task/Amplification Shader を使わなくてもよい理由を書け。

演習 1.2: 適用候補を選ぶ

次のうち、メッシュシェーダーを試す価値が高いものを選び、理由を 1 行ずつ書け。(a) 1 枚のフルスクリーン三角形、(b) 数百万三角形の都市、(c) 10 個の 2D アイコン、(d) 大量の小さな岩を距離 LOD 付きで描く場面。

GPU の実行モデルを理解する

2.1 CPU のスレッドと GPU のスレッドは何が違うか

CPU は少数の高機能コアで、複雑な分岐や低遅延の逐次処理を得意とする。GPU は非常に多数の軽量な実行要素で、同種の計算を大量データへ適用することを得意とする。シェーダーの 1 実行をここでは **invocation** と呼ぶ。複数 invocation をまとめたものが**ワークグループ**であり、ワークグループ内では共有メモリとバリアを使って協調できる。

ハードウェアはさらに、複数 invocation を **subgroup** として一緒に実行する。NVIDIA では warp、AMD では wave と呼ばれることが多い。ただし、サイズをアプリが決め打ちしてはいけない。API の subgroup 情報を問い合わせ、32、64、128 スレッドなど複数のワークグループ構成を実測する。

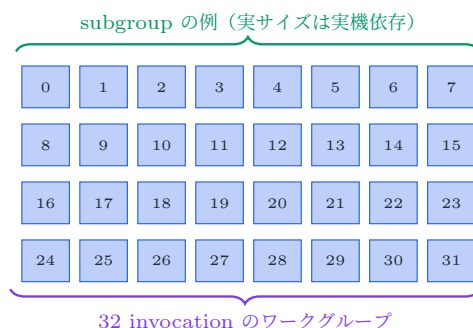


図 2.1: ワークグループと subgroup の関係を表す概念図

2.2 識別子を読み分ける

Mesh Shader のコードでは、次の識別子を頻繁に使う。

表 2.1: シェーダー内の代表的な識別子

意味	GLSL の例	HLSL の例
グループ内の 1 次元番号	<code>gl_LocalInvocationIndex</code>	<code>SV_GroupIndex</code>
グループ内の 3 次元番号	<code>gl_LocalInvocationID</code>	<code>SV_GroupThreadID</code>
ディスパッチ全体のグループ番号	<code>gl_WorkGroupID</code>	<code>SV_GroupID</code>
ディスパッチ全体のスレッド番号	組み合わせて計算	<code>SV_DispatchThreadID</code> (利用可能な段に注意)
subgroup 内番号	<code>gl_SubgroupInvocationID</code>	<code>WaveGetLaneIndex()</code>

たとえば 1 グループが 1 メッシュレットを担当するなら、`gl_WorkGroupID.x` がメッシュレット番号になる。グループ内番号 0 から $V-1$ が頂点を書き、番号 0 から $P-1$ がプリミティブを書く、という分担が最も単純である。

2.3 メモリ階層

表 2.2: GPU シェーダーから見た主な記憶領域

領域	共有範囲	おおまかな性質	Mesh Shader での用途
レジスタ	1 invocation	最も近いが有限	一時ベクトル、局所インデックス
共有メモリ	1 ワークグループ	高速、明示バリアが必要	メッシュレットの一時展開、可視数の圧縮
Storage/Structured Buffer	ディスパッチ全体	大容量、キャッシュ経由	頂点、三角形、境界、マテリアル
定数/Uniform Buffer	ディスパッチ全体	小さな読み取り値向け	カメラ行列、視錐台平面
Mesh 出力	1 ワークグループ	ラスタライザへ渡す専用領域	出力頂点、出力プリミティブ

共有メモリを使えば、全員で頂点を読み込み、再利用できる。しかし、共有メモリを増やしすぎると同時常駐できるワークグループ数が減る。また、出力配列の宣言上限も内部資源を消費するため、「実

際は 3 頂点しか出さないのに最大 256 頂点で宣言する」といった設計は避ける。AMD と NVIDIA の公式最適化資料も、出力配列と payload を小さく保つことを勧めている [15, 17]。

2.4 出力上限は設計条件である

Direct3D 12 Mesh Shader の必須上限には、1 グループ最大 128 スレッド、最大 256 出力頂点、最大 256 出力プリミティブなどが含まれる [1]。Vulkan では `VkPhysicalDeviceMeshShaderPropertiesEXT` から、Task/Mesh 段ごとのワークグループ数・サイズ、出力頂点数、出力プリミティブ数、共有メモリ、payload などを問い合わせる。したがって本書の数値は候補プロファイルであり、実行前に必ず実機上限と照合する。

表 2.3: 学習用に比較するメッシュレットプロファイル

プロファイル	最大頂点	最大三角形	32 スレッド時の頂点反復	主な目的
S	32	32	1	実装確認、出力を小さく
M	64	64	2	汎用的な初期値
N 系候補	64	126	2	NVIDIA 資料・サンプルで多い候補
L	128	256	4	AMD 資料で検証候補、密なクスタ

64 頂点・126 三角形は NVIDIA の初期資料や複数サンプルでよく使われ、AMD 資料では 128 頂点・256 三角形も候補として比較される。これはメーカー固定の規則ではない。両社 GPU で S/M/N 系/L を計測し、出力利用率、カリング粒度、占有率、メモリアクセスの総合で選ぶ [16, 15]。

2.5 分岐とダイバージェンス

同じ subgroup 内で半分が if の真側、半分が偽側へ進むと、ハードウェアは両側を順に実行し、不要な lane を無効化する場合がある。これをダイバージェンスという。短い分岐は問題にならないが、複雑なマテリアル処理や長いループが lane ごとに異なると効率が落ちる。

メッシュレット分割は形状だけでなく、マテリアルや処理経路も揃えるとよい。同じメッシュレット内に透明・不透明、スキニング有無、異なるデコード方式を混在させない設計が基本である。

演習 2.1: 仕事の分担を設計する

最大 64 頂点・最大 126 三角形のメッシュレットを、32 スレッドで処理する。各スレッドが担当する頂点番号と三角形番号を、`for (i = lane; i < count; i += 32)` の形で表せ。最も多く処理するスレッドは、頂点と三角形をそれぞれ何個担当するか。

演習 2.2: 資源の無駄を見つける

常に最大 40 頂点しか含まないデータに対し、シェーダーが最大 256 頂点の出力配列を宣言している。正しさと性能を分けて、何が問題になり得るか説明せよ。

メッシュレットというデータ単位

3.1 大きなメッシュを小さく切る

メッシュレットは、頂点数とプリミティブ数に上限を設けた小さな局所メッシュである。大きなモデルを数十から数百三角形の塊へ分割し、各塊を 1 ワークグループで処理できるようにする。共有頂点を同じ塊へ集めると頂点読み込みを再利用でき、空間的に近い三角形を集めると境界球が小さくなってカリングしやすい。

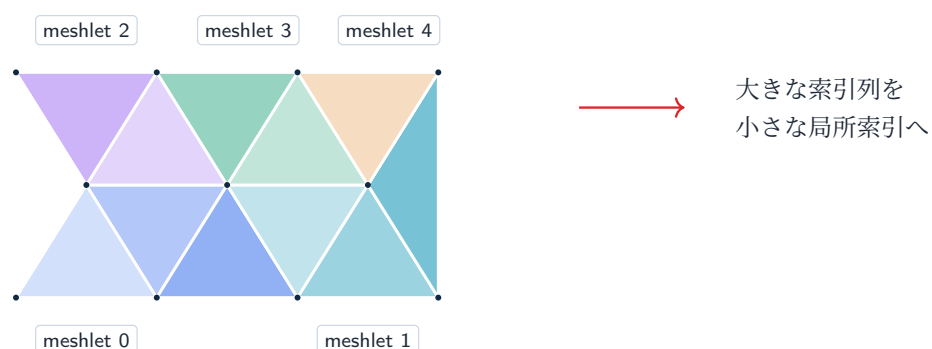


図 3.1: 連結した三角形群を複数メッシュレットへ分割する

3.2 2 段階のインデックス

メッシュレットでは、次の 2 段階参照を使うことが多い。

1. **頂点参照表**: メッシュレット局所頂点番号から、モデル全体の頂点番号へ変換する。
2. **局所三角形**: 0 から $V - 1$ の小さな局所番号で 3 頂点を指定する。

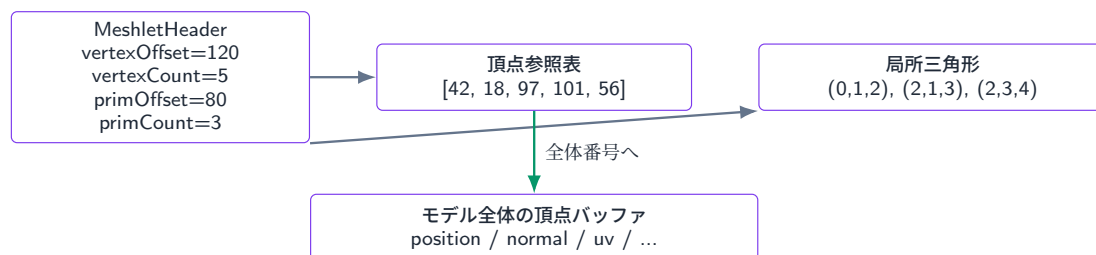


図 3.2: メッシュレットの代表的なデータ構造

C++ 側の最小構造は次のように表せる。

```

1 struct Meshlet {
2     std::uint32_t vertexOffset;
3     std::uint32_t vertexCount;
4     std::uint32_t primitiveOffset;
5     std::uint32_t primitiveCount;
6 };
7
8 // meshletVertexIndices[vertexOffset + localVertex]
9 // packedTriangles[primitiveOffset + localPrimitive]
  
```

Listing 3.1: 単純なメッシュレット記述子

最大局所頂点数が 256 以下なら、三角形の各局所番号は 8bit に収まる。3 個を 24bit へ詰め、残り 8bit をフラグに使う方式もある。ただし、シェーダーでの非境界整列読み出しや、API が要求するバッファアライメントを考え、32bit 単位で読む設計が扱いやすい。

```

1 std::uint32_t packTriangle(std::uint32_t a,
2                           std::uint32_t b,
3                           std::uint32_t c) {
4     assert(a < 256 && b < 256 && c < 256);
5     return a | (b << 8u) | (c << 16u);
6 }
  
```

Listing 3.2: 3つの 8bit 局所インデックスを 32bit へ格納する例

3.3 良い分割の条件

「最大数へ詰め込む」だけでは良いメッシュレットにならない。評価したい指標は次の通りである。

表 3.1: メッシュレット分割の代表的な評価軸

評価軸	良い状態	悪い状態
頂点再利用	共有辺の三角形が同じ塊に入り、局所頂点が少ない	離れた三角形を混ぜ、重複参照が多い
空間的まとまり	境界球が小さい	細長く散らばり、球が大きい
法線のまとまり	表面方向が似て、法線コーンが狭い	表裏や鋭角面を混ぜ、コーンが広い
充填率	上限近くまで有効頂点・三角形がある	小さな塊が大量にできる
マテリアル整合	同じパイプライン・材質で揃う	異なる分岐や descriptor を混在
LOD 対応	親子クラスタ関係と誤差を持つ	距離だけで全モデルを一括切替

すべてを同時に最適化するのは難しい。たとえば空間的まとまりを優先しすぎると、共有頂点の再利用が少し悪くなる場合がある。最終的には、オフライン変換時間よりランタイムの可視率、帯域、出力利用率を測る。

3.4 単純な貪欲分割

学習用には、次の貪欲法で十分である。

1. 未処理の三角形を 1 つ選び、新しいメッシュレットを作る。
2. 現在の境界辺に接する未処理三角形を候補にする。
3. 追加後の局所頂点数と三角形数が上限以内か調べる。
4. 新規頂点が少なく、境界球の増加が小さい候補を選ぶ。
5. 候補がない、または上限に達したら確定する。

候補 t の簡易スコアを、次のように置ける。

$$S(t) = w_v \Delta V(t) + w_r \Delta R(t) + w_n \Delta \theta(t), \quad (3.1)$$

ここで ΔV は追加局所頂点数、 ΔR は境界球半径の増分、 $\Delta \theta$ は法線コーン角の増分である。重み w_v, w_r, w_n を変えた複数データを生成し、実シーンで比較するとよい。本番では meshoptimizer などの実績あるライブラリを使う選択肢もあるが、まず自作してデータの意味を理解するとデバッグしやすい。

3.5 境界情報を添える

各メッシュレットに次を持たせると、描く前に安価な判定ができる。

- オブジェクト空間の境界球 ($\mathbf{c}, r$)
- 法線コーンの軸 $\mathbf{a}$ と開き角の余弦 $\cos \theta$
- LOD の幾何誤差 e
- マテリアル ID、LOD レベル、親クラスタ ID

```
1 struct MeshletBounds {  
2     float center[3];  
3     float radius;  
4     float coneAxis[3];  
5     float coneCutoff;    // cos(theta) など。定義を明文化する  
6     float lodError;  
7     std::uint32_t materialId;  
8 };
```

Listing 3.3: 境界情報を含む記述子の一例

注意

法線コーンの `coneCutoff` は、ライブラリごとに「角度」「cos」「符号を反転した cos」など定義が異なる。データ生成側とシェーダー側で式を 1 か所にまとめ、正面・背面・斜めのテストモデルを用意する。

演習 3.1: 局所インデックスを作る

全体頂点番号を使う三角形が、 $(42, 18, 97)$, $(97, 18, 101)$, $(97, 101, 56)$ である。初出順に頂点参照表を作り、3 個の局所三角形へ変換せよ。

演習 3.2: 格納量を比べる

64 頂点・126 三角形のメッシュレットを考える。頂点参照を 32bit、局所三角形を (a) 各成分 32bit、(b) 各成分 8bit として格納すると、接続情報は何 byte か。ヘッダ 16byte を加えた場合も計算せよ。

第 4

カリングと LOD に必要な数学

4.1 座標をベクトルとして扱う

3次元の位置や方向は、成分を並べたベクトルで表す。

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \|\mathbf{p}\| = \sqrt{x^2 + y^2 + z^2}. \quad (4.1)$$

位置同士の差 $\mathbf{q} - \mathbf{p}$ は方向を表す。方向を長さ 1 へそろえる操作が正規化である。

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}, \quad \|\mathbf{v}\| \neq 0. \quad (4.2)$$

ゼロベクトルを正規化すると NaN が生じるため、長さの二乗が小さい場合のフォールバックを用意する。GPU では NaN が後段へ広がり、三角形全体が消えるなど見つけにくい症状になる。

4.2 内積は「向きの近さ」を測る

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta. \quad (4.3)$$

両方が単位ベクトルなら、内積は $\cos \theta$ そのものになる。同じ向きで 1、直交で 0、反対向きで -1 である。この性質を使い、面の向きとカメラ方向が十分反対なら、メッシュレット全体を背面として捨てられる。

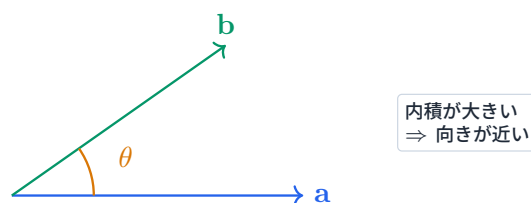


図 4.1: 内積と 2 つの方向のなす角

4.3 外積は面の法線を作る

三角形の頂点を $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ とすると、未正規化法線は

$$\mathbf{n} = (\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0) \quad (4.4)$$

である。頂点順が反転すると法線も反転する。左手系・右手系よりも、**頂点の巻き順、API の front face 設定、行列の規約を一貫させることが重要である**。面積がほぼ 0 の退化三角形では $\|\mathbf{n}\|$ が小さくなり、正規化できない。アセット変換時に除去または明示的に扱う。

4.4 同次座標と行列

平行移動を行列に含めるため、3 次元位置を 4 成分 $(x, y, z, 1)$ で表す。方向は $(x, y, z, 0)$ である。オブジェクトからクリップ空間までの変換を、列ベクトル規約なら次のように書ける。

$$\mathbf{p}_{clip} = P V M \mathbf{p}_{object}. \quad (4.5)$$

M はモデル行列、 V はビュー行列、 P は射影行列である。HLSL で $\text{mul}(\mathbf{v}, M)$ と書くか $\text{mul}(M, \mathbf{v})$ と書くか、GLSL で列ベクトルを使うかにより、CPU 側の転置要否が変わる。「行優先か列優先か」だけでなく、**ベクトルを左から掛けるか右から掛けるか**まで決め、テストに固定する。

透視除算後の正規化デバイス座標は

$$\mathbf{p}_{ndc} = \left(\frac{x_c}{w_c}, \frac{y_c}{w_c}, \frac{z_c}{w_c} \right) \quad (4.6)$$

である。Vulkan と Direct3D では深度範囲や Y 方向の扱いを実装設定と合わせる必要がある。GLSL/HLSL を共有する場合は、射影行列を共通化するのか、シェーダーで補正するのかを一つに決める。

表 4.1: 座標系で明文化すべき項目

項目	プロジェクトで決める内容
手系	ワールド・ビュー空間を左手系または右手系のどちらで扱うか
行列規約	行ベクトル/列ベクトル、CPU メモリ配置、シェーダーの mul 順
前面	時計回り/反時計回りのどちらを front face とするか
NDC 深度	使用 API と射影行列が期待する範囲
画面 Y	viewport の符号反転、射影行列補正、シェーダー補正のどれを使うか
法線変換	非一様スケール時に逆転置行列を使うか

4.5 境界球と視錐台平面

平面を単位法線 $\mathbf{n}$ と定数 d で

$$f(\mathbf{x}) = \mathbf{n} \cdot \mathbf{x} + d = 0 \quad (4.7)$$

と表す。内側を $f(\mathbf{x}) \geq 0$ と定義する。中心 $\mathbf{c}$ 、半径 r の球が平面の完全な外側にある条件は

$$\mathbf{n} \cdot \mathbf{c} + d < -r \quad (4.8)$$

である。視錐台の 6 平面のうち 1 つでもこの条件を満たせば、その球は見えない。

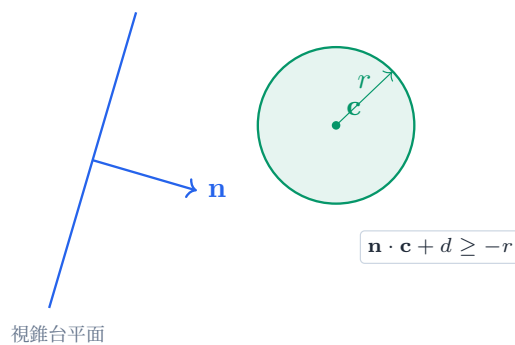


図 4.2: 平面と境界球の判定。法線が内側を向く規約の例

モデル行列に非一様スケールが含まれる場合、中心は行列で変換できるが、半径は単純に 1 つの軸の倍率を掛けてはいけない。安全側には、モデル行列の 3 本の基底ベクトル長の最大値を s_{max} とし

て $r' = s_{max}r$ とする。より正確な楕円体判定は高価なので、まず保守的な球を使う。

4.6 法線コーンによる背面カリング

メッシュレット内の面法線が、軸 $\mathbf{a}$ を中心に半角 θ 以内へ収まるとする。カメラへ向かう単位方向を $\mathbf{v}$ とする。全ての面が背面になる条件は、コーンの定義と頂点巻き順で符号が変わる。概念的には、 $\mathbf{a} \cdot \mathbf{v}$ と閾値を比較し、カメラ方向がコーンの反対側へ十分離れているかを見る。

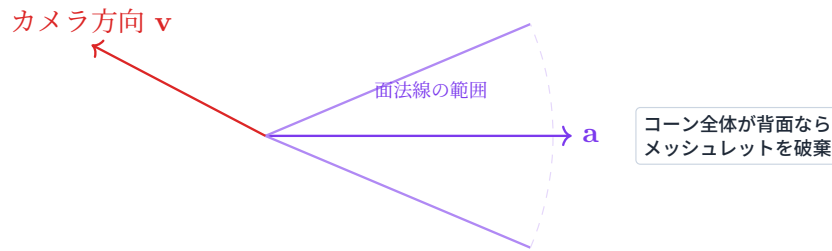


図 4.3: 法線コーンの概念。実装前に符号と方向をテストする

カメラが境界球の内部にいる場合、単一の中心方向で判定すると誤ることがある。また、両面材質、葉、布、開いた薄板では法線コーンカリングを無効にする。境界球の接線を考慮した保守的判定を使うか、最初は視錐台カリングだけを実装して段階的に追加する。

4.7 画面上の誤差と LOD

幾何誤差 e がビュー空間でどれだけの画素に見えるかを概算する。垂直視野角を f_y 、画面高さを H 、カメラからの距離を $z > 0$ とすると、投影スケールは

$$s = \frac{H}{2 \tan(f_y/2)}, \quad e_{px} \approx \frac{e s}{z}. \quad (4.9)$$

e_{px} が許容閾値 τ より小さければ粗い LOD を選べる。距離だけで切り替えるより、解像度と FOV を含むため、異なる画面でも見た目を揃えやすい。

切り替え境界で LOD が往復する場合は、**ヒステリシス**を入れる。粗くする閾値を 1.0px、細かく戻す閾値を 1.3px のように分ける。急な切り替えが目立つ場合は、一定期間のディザやクロスフェードを使うが、重複描画コストも測る。

数学の補助線

境界球半径の画面上サイズも同様に $r_{px} \approx rs/z$ で概算できる。非常に小さいメッシュレットをまとめて捨てる small primitive culling や、LOD 選択の補助に使える。ただし、 z が 0 付近やカメラ背後では式が不安定なので、near plane と符号を先に判定する。

演習 4.1: 球と平面

平面 $\mathbf{n} = (1, 0, 0)$, $d = 2$ について、中心 $(-3.2, 0, 0)$ 、半径 0.5 の球は完全に外側か。中心を $(-2.3, 0, 0)$ へ変えた場合も判定せよ。内側を $\mathbf{n} \cdot \mathbf{x} + d \geq 0$ とする。

演習 4.2: 画面誤差

画面高さ 1080px、垂直 FOV60 度、幾何誤差 0.02m、距離 10m とする。投影誤差 e_{px} を概算せよ。距離が半分になると何倍か。

演習 4.3: 座標規約のテストを作る

単位三角形、X 軸 90 度回転、負スケール、非一様スケールの 4 ケースについて、画面に期待する前面方向と法線を表にせよ。この表が、行列転置・front face・法線変換の回帰テストになる理由を説明せよ。

第Ⅱ部

動かして理解する

開発環境と機能検出

5.1 どの経路で学ぶか

表 5.1: 本書で扱う実装経路

経路	OS	シェーダー	向いている用途
Vulkan + GLSL	Windows / Linux	GLSL → SPIR-V	API 仕様を直接学ぶ、 Khronos Samples を使う
Vulkan + HLSL	Windows / Linux	HLSL/DXC SPIR-V	→ シェーダー言語を HLSL へ統一する
D3D12 + HLSL	Windows	HLSL/DXC DXIL	→ Windows 専用エンジン、 DirectX サンプルを使う

NVIDIA か Radeon かでソースを分けるのではなく、API の機能問い合わせで経路を選ぶ。GPU 名の文字列判定は、同じ製品でもドライバや仮想環境で変わり、将来の製品を誤判定する。メーカー名は性能ログの分類に使い、正しさは拡張・feature・property で決める。

5.2 必要な道具

チェックリスト

- 64bit C++ コンパイラと CMake
- Vulkan を使う場合は Vulkan SDK、検証レイヤ、SPIR-V コンパイラ
- Direct3D 12 を使う場合は Windows SDK と DXC
- Mesh Shader 対応 GPU と、対応機能を公開するドライバ
- GPU キャプチャまたはプロファイラ
- Git と、公式サンプルを取得できる環境

Windows では Visual Studio の x64 Native Tools Command Prompt または PowerShell を使う。Linux ではディストリビューションの C++ コンパイラ、CMake、X11/Wayland 開発パッケージを用意する。ドライバの「新しさ」だけを信用せず、後述のコードで VK_EXT_mesh_shader または D3D12 Mesh Shader Tier を確認する。

5.3 公式 Vulkan Samples をビルドする

Khronos の Vulkan Samples には、最小のメッシュシェーディング例とメッシュレットカリング例が含まれる [9, 10]。リポジトリの構成や依存関係は更新されるため、付属の build 文書を優先する。代表的なコマンドは次の通りである [12]。

ハンズオン: Windows で Vulkan Samples をビルド

リポジトリ直下で実行する。

```
1 cmake -G "Visual Studio 17 2022" -A x64 `
2     -S . -B build/windows
3 cmake --build build/windows --config Release `
4     --target vulkan_samples
5
6 build\windows\app\bin\Release\AMD64\vulkan_samples.exe `
7     sample mesh_shading
```

サンプル名が変わった場合は、実行ファイルへ `-help` を渡して一覧を確認する。

ハンズオン: Linux で Vulkan Samples をビルド

```
1 cmake -G "Unix Makefiles" -B build/linux `
2     -DCMAKE_BUILD_TYPE=Release
3 cmake --build build/linux --config Release `
4     --target vulkan_samples -j"$(nproc)"
5
6 ./build/linux/app/bin/Release/x86_64/vulkan_samples --help
```

Wayland を明示する場合は、構成時に `-DVKB_WSI_SELECTION=WAYLAND` を追加する。

5.4 Vulkan で拡張を確認する

最初に device extension 一覧から名前を確認し、その後 features と properties を問い合わせる。

```

1  bool hasDeviceExtension(VkPhysicalDevice physicalDevice,
2                          const char* wanted) {
3      std::uint32_t count = 0;
4      vkEnumerateDeviceExtensionProperties(
5          physicalDevice, nullptr, &count, nullptr);
6
7      std::vector<VkExtensionProperties> props(count);
8      vkEnumerateDeviceExtensionProperties(
9          physicalDevice, nullptr, &count, props.data());
10
11     return std::any_of(props.begin(), props.end(),
12                        [wanted](const VkExtensionProperties& p) {
13                            return std::strcmp(p.extensionName, wanted) == 0;
14                        });
15 }
16
17 const bool hasMeshExtension = hasDeviceExtension(
18     physicalDevice, VK_EXT_MESH_SHADER_EXTENSION_NAME);

```

Listing 5.1: Vulkan device extension の確認

拡張名があっても、Task Shader と Mesh Shader の feature を個別に見る。Mesh Shader だけが
必要なら taskShader を必須にしない。

```

1  VkPhysicalDeviceMeshShaderFeaturesEXT meshFeatures{
2      VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MESH_SHADER_FEATURES_EXT
3  };
4  VkPhysicalDeviceFeatures2 features2{
5      VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_FEATURES_2
6  };
7  features2.pNext = &meshFeatures;
8
9  if (hasMeshExtension) {
10     vkGetPhysicalDeviceFeatures2(physicalDevice, &features2);
11 }
12
13 VkPhysicalDeviceMeshShaderPropertiesEXT meshProps{
14     VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MESH_SHADER_PROPERTIES_EXT
15 };
16 VkPhysicalDeviceProperties2 properties2{
17     VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_PROPERTIES_2
18 };

```

```

19 properties2.pNext = &meshProps;
20
21 if (hasMeshExtension) {
22     vkGetPhysicalDeviceProperties2(physicalDevice, &properties2);
23 }
24
25 const bool canUseMesh = hasMeshExtension &&
26                         meshFeatures.meshShader == VK_TRUE;
27 const bool canUseTask = canUseMesh &&
28                         meshFeatures.taskShader == VK_TRUE;

```

Listing 5.2: Vulkan feature と property の問い合わせ

最低でも次の値をログへ出す。

- maxMeshWorkGroupInvocations
- maxMeshOutputVertices と maxMeshOutputPrimitives
- maxMeshSharedMemorySize
- maxTaskWorkGroupInvocations と maxTaskPayloadSize
- prefersLocalInvocationVertexOutput などの preference

有効化時は extension 名を device creation へ追加し、feature 構造を pNext へ接続する。問い合わせに使った構造をそのまま渡すのではなく、利用する field だけ VK_TRUE にした構造を作ると意図が明確になる。

```

1  VkPhysicalDeviceMeshShaderFeaturesEXT enableMesh{
2      VK_STRUCTURE_TYPE_PHYSICAL_DEVICE_MESH_SHADER_FEATURES_EXT
3  };
4  enableMesh.meshShader = VK_TRUE;
5  enableMesh.taskShader = requestTaskShader ? VK_TRUE : VK_FALSE;
6
7  enabledExtensions.push_back(
8      VK_EXT_MESH_SHADER_EXTENSION_NAME);
9
10 deviceCreateInfo.enabledExtensionCount =
11     static_cast<std::uint32_t>(enabledExtensions.size());
12 deviceCreateInfo.ppEnabledExtensionNames =
13     enabledExtensions.data();
14 deviceCreateInfo.pNext = &enableMesh;

```

Listing 5.3: Vulkan device 作成時の有効化

関数ポインタを自前で読む場合は、device 作成後に取得し、null でないことを確認する。Volk などの loader を使う場合も、拡張有効化の責任はアプリ側にある。

```
1 auto cmdDrawMeshTasks = reinterpret_cast<PFN_vkCmdDrawMeshTasksEXT>(  
2     vkGetDeviceProcAddr(device, "vkCmdDrawMeshTasksEXT"));  
3 if (!cmdDrawMeshTasks) {  
4     throw std::runtime_error(  
5         "vkCmdDrawMeshTasksEXT is not available");  
6 }
```

Listing 5.4: 描画コマンド関数の取得例

5.5 Direct3D 12 で Tier を確認する

D3D12 では Options7 の feature data を問い合わせる。識別子は D3D12_FEATURE_D3D12_OPTIONS7 である。結果の Mesh Shader Tier が D3D12_MESH_SHADER_TIER_NOT_SUPPORTED なら、従来経路へ切り替える [3]。

```
1 D3D12_FEATURE_DATA_D3D12_OPTIONS7 options7{};  
2 const HRESULT hr = device->CheckFeatureSupport(  
3     D3D12_FEATURE_D3D12_OPTIONS7,  
4     &options7,  
5     sizeof(options7));  
6  
7 const bool canUseMesh = SUCCEEDED(hr) &&  
8     options7.MeshShaderTier !=  
9     D3D12_MESH_SHADER_TIER_NOT_SUPPORTED;
```

Listing 5.5: D3D12 Mesh Shader Tier の確認

シェーダーコンパイラについては、Mesh Shader が Shader Model 6.5 の ms_6_5、Amplification Shader が as_6_5 から導入された [2]。実行時機能と、使用する DXC が対象 profile を受理することの両方を確認する。

5.6 フォールバックを最初から設計する

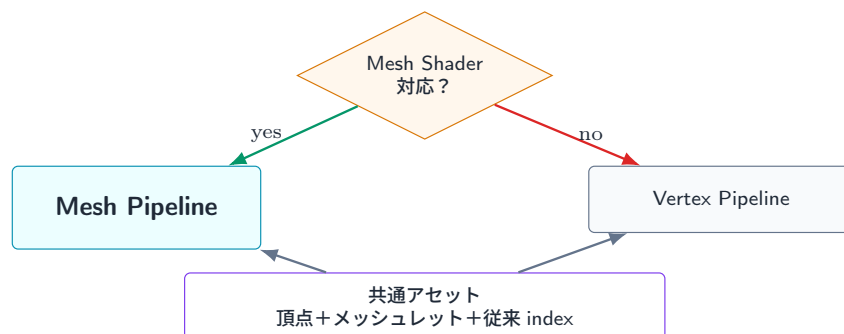


図 5.1: 実行時機能検出によるパイプライン選択

アセット変換時に、メッシュレットから従来 index buffer を再構成できるようにしておく。DirectX の仕様も、同じメッシュレットを通常 index buffer へ変換して Vertex Shader fallback に利用できると説明している [1]。次の 3 段階を用意すると、古い GPU だけでなくバグ切り分けにも役立つ。

1. Mesh + Task/Amplification
2. Mesh only
3. Vertex + Index

演習 5.1: 機能ログを設計する

起動時に 1 回だけ出力する機能ログの項目を 10 個考えよ。GPU 名だけでなく、Mesh/Task 対応、出力上限、ワークグループ上限、subgroup サイズ、使用 API 経路を含めること。

演習 5.2: フォールバックの責任範囲

Mesh Shader 非対応でも描画を続行するアプリを考える。シェーダー、PSO、アセット、描画コマンド、性能統計の 5 観点で、何を二重化し何を共有するか表にせよ。

最初の三角形を出力する

6.1 GLSL の Mesh Shader

最初は vertex buffer を使わず、シェーダー内の定数から三角形を 1 枚生成する。32 invocation を起動するが、頂点を書くのは lane 0 から 2、三角形を書くのは lane 0 だけである。SetMeshOutputsEXT は uniform control flow で呼び、出力への書き込みより前に置く [5]。

```
1  #version 460
2  #extension GL_EXT_mesh_shader : require
3
4  layout(local_size_x = 32) in;
5  layout(triangles, max_vertices = 3,
6         max_primitives = 1) out;
7
8  layout(location = 0) out vec3 outColor[];
9
10 const vec2 positions[3] = vec2[](
11     vec2( 0.0, -0.65),
12     vec2( 0.65, 0.55),
13     vec2(-0.65, 0.55)
14 );
15
16 const vec3 colors[3] = vec3[](
17     vec3(1.0, 0.2, 0.2),
18     vec3(0.2, 1.0, 0.3),
19     vec3(0.2, 0.4, 1.0)
20 );
21
22 void main() {
23     SetMeshOutputsEXT(3, 1);
24
25     uint lane = gl_LocalInvocationIndex;
26     if (lane < 3) {
27         gl_MeshVerticesEXT[lane].gl_Position =
28             vec4(positions[lane], 0.0, 1.0);
```

```
29     outColor[lane] = colors[lane];
30 }
31
32 if (lane == 0) {
33     gl_PrimitiveTriangleIndicesEXT[0] =
34         uvec3(0, 1, 2);
35 }
36 }
```

Listing 6.1: triangle.mesh: Vulkan GLSL の最小 Mesh Shader

Fragment Shader は通常通りである。

```
1 #version 460
2
3 layout(location = 0) in vec3 inColor;
4 layout(location = 0) out vec4 outColor;
5
6 void main() {
7     outColor = vec4(inColor, 1.0);
8 }
```

Listing 6.2: triangle.frag: 頂点色を補間する Fragment Shader

SPIR-V へコンパイルする。

```
1 glslangValidator -V --target-env vulkan1.2 \
2     -S mesh triangle.mesh -o triangle.mesh.spv
3 glslangValidator -V --target-env vulkan1.2 \
4     -S frag triangle.frag -o triangle.frag.spv
```

注意

SetMeshOutputsEXT の引数は、宣言した max_vertices と max_primitives 以下でなければならない。出力配列は write-only であり、書いた値を同じ Mesh Shader から読み返す設計にしない。また、三角形インデックスはベクトル全体を一度に書く。

6.2 Vulkan の graphics pipeline

Mesh Shader を使う graphics pipeline では、vertex stage を Mesh stage へ置き換える。Task Shader を使わない最小構成は、Mesh + Fragment である。Vertex input binding や attribute は空にし、描画時に vertex buffer や index buffer を bind しない。

```
1  VkPipelineShaderStageCreateInfo stages[2]{};
2
3  stages[0].sType =
4      VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
5  stages[0].stage = VK_SHADER_STAGE_MESH_BIT_EXT;
6  stages[0].module = meshModule;
7  stages[0].pName = "main";
8
9  stages[1].sType =
10     VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
11 stages[1].stage = VK_SHADER_STAGE_FRAGMENT_BIT;
12 stages[1].module = fragmentModule;
13 stages[1].pName = "main";
14
15 VkPipelineVertexInputStateCreateInfo vertexInput{
16     VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO
17 };
18 VkPipelineInputAssemblyStateCreateInfo inputAssembly{
19     VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO
20 };
21 inputAssembly.topology =
22     VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;
```

Listing 6.3: Vulkan shader stage 構成の要点

実際の pipeline 作成には、viewport、rasterization、multisample、depth/stencil、color blend、render pass または dynamic rendering 情報も必要である。これらは Hello Triangle と同じであり、Mesh Shader 固有なのは stage と入力アセンブリの使われ方である。

描画コマンドは 1 個の Mesh Shader ワークグループを起動する。

```
1  vkCmdBindPipeline(commandBuffer,
2                      VK_PIPELINE_BIND_POINT_GRAPHICS,
3                      meshPipeline);
4
5  cmdDrawMeshTasks(commandBuffer, 1, 1, 1);
```

Listing 6.4: Vulkan で 1 ワークグループを起動

6.3 HLSL の Mesh Shader

Direct3D 12 版も同じ分担である。Mesh Shader の出力配列は関数引数に宣言し、SV_Position を含む頂点構造を用意する。

```
1 struct VertexOut {
2     float4 position : SV_Position;
3     float3 color    : COLOR0;
4 };
5
6 static const float2 kPositions[3] = {
7     float2( 0.0, -0.65),
8     float2( 0.65, 0.55),
9     float2(-0.65, 0.55)
10 };
11
12 static const float3 kColors[3] = {
13     float3(1.0, 0.2, 0.2),
14     float3(0.2, 1.0, 0.3),
15     float3(0.2, 0.4, 1.0)
16 };
17
18 [outputtopology("triangle")]
19 [numthreads(32, 1, 1)]
20 void MSMain(
21     uint lane : SV_GroupIndex,
22     out vertices VertexOut verts[3],
23     out indices uint3 triangles[1]) {
24
25     SetMeshOutputCounts(3, 1);
26
27     if (lane < 3) {
28         verts[lane].position =
29             float4(kPositions[lane], 0.0, 1.0);
30         verts[lane].color = kColors[lane];
31     }
32
33     if (lane == 0) {
34         triangles[0] = uint3(0, 1, 2);
35     }
36 }
37
```

```

38 float4 PSMain(VertexOut input) : SV_Target0 {
39     return float4(input.color, 1.0);
40 }

```

Listing 6.5: triangle.hlsl: D3D12 の最小 Mesh Shader と Pixel Shader

DXIL へコンパイルする。

```

1 dxc -T ms_6_5 -E MSMain -Fo triangle.ms.dxil triangle.hlsl
2 dxc -T ps_6_5 -E PSMain -Fo triangle.ps.dxil triangle.hlsl

```

D3D12 の pipeline state は、Mesh Shader 対応の pipeline state stream を使う。VS/HS/DS/GS を同時に指定せず、Input Layout も指定しない [1]。コマンドリストは Mesh Shader 対応 interface から次を呼ぶ。

```
commandList6->DispatchMesh(1, 1, 1);
```

6.4 同じ HLSL を Vulkan へ使う

DXC の SPIR-V backend を使うと、HLSL から SPIR-V を生成できる。メッシュシェーダーでは、使用する DXC が SPV_EXT_mesh_shader へ対応しているか確認する。代表的な形は次の通りである。

```

1 dxc -spirv -T ms_6_5 -E MSMain `
2   -fspv-target-env=vulkan1.2 `
3   -fspv-extension=SPV_EXT_mesh_shader `
4   -Fo triangle.mesh.spv triangle.hlsl

```

binding 規則、行列レイアウト、GLSL との組み込み変数差を吸収するため、API 共通ヘッダを作る。コンパイラ引数は CI で固定し、DXIL 用と SPIR-V 用を両方コンパイルして警告をエラー扱いにする。

6.5 画面が出ないときの順序

1. 機能検出ログで Mesh Shader 経路へ入ったか。
2. シェーダーコンパイルと validation が成功したか。
3. pipeline に Mesh stage が入り、Vertex stage が混在していないか。
4. cull mode と front face を一時的に無効化したか。

5. viewport/scissor、render target、color write mask が正しいか。
6. SetMeshOutputsEXT/SetMeshOutputCounts が出力書き込みを支配しているか。
7. 三角形インデックスが 0,1,2 で、全頂点を書いているか。

ハンズオン: 三角形を観察する

まず cull mode を none、depth test を off、wireframe が使える環境では wireframe にする。GPU キャプチャで、Mesh Shader が 1 グループ起動され、3 頂点・1 プリミティブを出していることを確認する。次に front face を反転し、back-face culling を有効にして、三角形が消える側を記録する。

演習 6.1: 2 枚の三角形へ増やす

4 頂点・2 三角形で四角形を作れ。局所頂点 0,1,2,3 を使い、2 個の uvec3/uint3 を出力する。2 枚の巻き順が一致するようにせよ。

演習 6.2: lane を可視化する

頂点色を lane 番号から計算し、lane 0,1,2 が異なる色になるようにせよ。なぜ Fragment/Pixel Shader では三角形内部が滑らかに混色されるのか、補間という語を使って説明せよ。

第 7

バッファからメッシュレットを描く

7.1 データレイアウトを固定する

最初の実データ版では、位置を `vec4/float4`、メッシュレット記述子を `uint` 4 個、局所三角形を 32bit packed 値とする。圧縮は後回しにし、CPU と GPU の構造が同じ 16byte 単位になるようにする。

表 7.1: 学習用バッファ構成

バッファ	要素	要素サイズ	役割
Positions	float4	16B	モデル全体の位置
Meshlets	uint4 相当	16B	offset と count
MeshletVertexIndices	uint	4B	局所頂点から全体頂点への参照
PackedTriangles	uint	4B	3 個の 8bit 局所番号
Objects	行列など	16B 倍数	world と view-projection

ベンダー横断の考え方

最初から 12byte の `float3` を構造体配列に詰めると、C++、GLSL `std430`、HLSL `Structured-Buffer` で `stride` の理解がずれやすい。学習段階では `float4` を使い、正しい画像を得てから帯域最適化する。

7.2 GLSL で読み出す

次のシェーダーは、1 ワークグループが 1 メッシュレットを担当する。最大 64 頂点・126 三角形を宣言し、32 invocation が `stride loop` で分担する。

```

1 #version 460
2 #extension GL_EXT_mesh_shader : require

```

```
3
4 layout(local_size_x = 32) in;
5 layout(triangles, max_vertices = 64,
6         max_primitives = 126) out;
7
8 struct Meshlet {
9     uint vertexOffset;
10    uint vertexCount;
11    uint primitiveOffset;
12    uint primitiveCount;
13 };
14
15 layout(set = 0, binding = 0, std430) readonly buffer Meshlets {
16     Meshlet meshlets[];
17 };
18 layout(set = 0, binding = 1, std430) readonly buffer Positions {
19     vec4 positions[];
20 };
21 layout(set = 0, binding = 2, std430) readonly buffer VertexRefs {
22     uint meshletVertexIndices[];
23 };
24 layout(set = 0, binding = 3, std430) readonly buffer Triangles {
25     uint packedTriangles[];
26 };
27 layout(set = 0, binding = 4) uniform Camera {
28     mat4 worldViewProj;
29 };
30
31 layout(location = 0) out vec3 outColor[];
32
33 uvec3 unpackTriangle(uint packed) {
34     return uvec3( packed & 0xffu,
35                 (packed >> 8u) & 0xffu,
36                 (packed >> 16u) & 0xffu);
37 }
38
39 vec3 debugColor(uint meshletId) {
40     uint h = meshletId * 1664525u + 1013904223u;
41     return vec3(float((h >> 0u) & 255u),
42                float((h >> 8u) & 255u),
43                float((h >> 16u) & 255u)) / 255.0;
44 }
```



```

45
46 void main() {
47     uint meshletId = gl_WorkGroupID.x;
48     uint lane = gl_LocalInvocationIndex;
49     Meshlet m = meshlets[meshletId];
50
51     SetMeshOutputsEXT(m.vertexCount, m.primitiveCount);
52
53     for (uint v = lane; v < m.vertexCount; v += 32u) {
54         uint globalVertex =
55             meshletVertexIndices[m.vertexOffset + v];
56         vec3 p = positions[globalVertex].xyz;
57         gl_MeshVerticesEXT[v].gl_Position =
58             worldViewProj * vec4(p, 1.0);
59         outColor[v] = debugColor(meshletId);
60     }
61
62     for (uint p = lane; p < m.primitiveCount; p += 32u) {
63         uint packed = packedTriangles[m.primitiveOffset + p];
64         gl_PrimitiveTriangleIndicesEXT[p] =
65             unpackTriangle(packed);
66     }
67 }

```

Listing 7.1: meshlet.mesh: バッファ駆動の GLSL Mesh Shader

各 lane が異なる出力要素へ書くため、頂点出力とプリミティブ出力の間に barrier は不要である。Mesh Shader 完了後にラスタライザが出力を読む。ただし、共有メモリへ一度展開して別 lane が読む設計へ変えた場合は、書き込み後に barrier() が必要になる。

CPU 側はメッシュレット数を X 方向のグループ数として渡す。1 回のコマンドに許されるグループ数上限を properties で確認し、超える場合は複数回に分けるか indirect 経路を使う。

```

1 const std::uint32_t meshletCount =
2     static_cast<std::uint32_t>(meshlets.size());
3 cmdDrawMeshTasks(commandBuffer, meshletCount, 1, 1);

```

Listing 7.2: 全メッシュレットを直接起動する

7.3 HLSL 版の対応

```
1  #define THREADS    32
2  #define MAX_VERTS  64
3  #define MAX_PRIMS  126
4
5  struct Meshlet {
6      uint vertexOffset;
7      uint vertexCount;
8      uint primitiveOffset;
9      uint primitiveCount;
10 };
11 struct VertexOut {
12     float4 position : SV_Position;
13     float3 color    : COLOR0;
14 };
15
16 StructuredBuffer<Meshlet> Meshlets : register(t0);
17 StructuredBuffer<float4> Positions : register(t1);
18 StructuredBuffer<uint> VertexRefs : register(t2);
19 StructuredBuffer<uint> PackedTriangles : register(t3);
20
21 cbuffer Camera : register(b0) {
22     float4x4 WorldViewProj;
23 };
24
25 uint3 UnpackTriangle(uint packed) {
26     return uint3( packed      & 0xff,
27                 (packed >> 8) & 0xff,
28                 (packed >> 16) & 0xff);
29 }
30
31 [outputtopology("triangle")]
32 [numthreads(THREADS, 1, 1)]
33 void MSMain(
34     uint lane : SV_GroupIndex,
35     uint3 groupId : SV_GroupID,
36     out vertices VertexOut verts[MAX_VERTS],
37     out indices uint3 triangles[MAX_PRIMS]) {
38
39     Meshlet m = Meshlets[groupId.x];
40     SetMeshOutputCounts(m.vertexCount, m.primitiveCount);
41
42     for (uint v = lane; v < m.vertexCount; v += THREADS) {
```

```

43     uint globalVertex = VertexRefs[m.vertexOffset + v];
44     float3 p = Positions[globalVertex].xyz;
45     verts[v].position = mul(WorldViewProj, float4(p, 1.0));
46     verts[v].color = frac(float3(groupId.x) *
47                           float3(0.37, 0.61, 0.83));
48 }
49
50 for (uint p = lane; p < m.primitiveCount; p += THREADS) {
51     triangles[p] =
52         UnpackTriangle(PackedTriangles[m.primitiveOffset + p]);
53 }
54 }

```

Listing 7.3: meshlet.hlsl: バッファ読み出しの要点

注意

GLSL と HLSL で同じ行列バイト列を共有する場合、mul の順序と matrix packing を必ずテストする。三角形は出るが回転や平行移動だけ壊れる場合、Mesh Shader ではなく行列規約が原因であることが多い。

7.4 CPU で最小データを作る

四角形を 2 三角形、4 頂点として、1 個のメッシュレットへ入れる。

```

1  std::vector<std::array<float, 4>> positions = {
2      {-0.6f, -0.6f, 0.0f, 1.0f},
3      { 0.6f, -0.6f, 0.0f, 1.0f},
4      { 0.6f,  0.6f, 0.0f, 1.0f},
5      {-0.6f,  0.6f, 0.0f, 1.0f},
6  };
7
8  std::vector<std::uint32_t> vertexRefs = {0, 1, 2, 3};
9  std::vector<std::uint32_t> triangles = {
10     packTriangle(0, 1, 2),
11     packTriangle(0, 2, 3),
12 };
13 std::vector<Meshlet> meshlets = {
14     Meshlet{0, 4, 0, 2}
15 };

```

Listing 7.4: 四角形 1 メッシュレットの学習用データ

GPU buffer への upload は通常の storage/structured buffer と同じである。Vulkan では usage に `VK_BUFFER_USAGE_STORAGE_BUFFER_BIT`、D3D12 では SRV として読める resource を用意する。転送完了前に描画しないよう、upload と shader read の同期を入れる。

ハンズオン: メッシュレット色分けビュー

複数メッシュレットを持つモデルを読み、`debugColor(meshletId)` で色分けする。同じ色の領域が空間的にまとまっているか、細長すぎないか、1 三角形だけの塊が多くないか観察する。画面にメッシュレット数、平均頂点数、平均三角形数、充填率を表示する。

演習 7.1: 境界外アクセスを防ぐ

CPU データが壊れ、`vertexCount=70` なのにシェーダー宣言は `max_vertices=64` だったとする。GPU へ送る前に行う validation を列挙し、エラー表示に含める ID と値を設計せよ。

演習 7.2: 頂点再利用率を求める

64 頂点・126 三角形のメッシュレットについて、三角形の頂点参照回数は 378 回である。単純な指標 $R = 378/64$ を計算せよ。同じ 126 三角形で 126 頂点なら何が変わるか。この指標だけでは分からない性能要因も 2 つ挙げよ。

第 8

Task/Amplification Shader で可視仕事を圧縮する

8.1 なぜ前段があるのか

Mesh Shader 内で不可視なら 0 出力にするだけでも正しい。しかし不可視メッシュレットが多いと、Mesh Shader ワークグループ自体は起動され、descriptor 読み込みや境界判定を行う。Task/Amplification Shader は、複数候補を 1 グループで調べ、可視な ID だけを payload へ詰め、その数だけ Mesh Shader を起動できる。

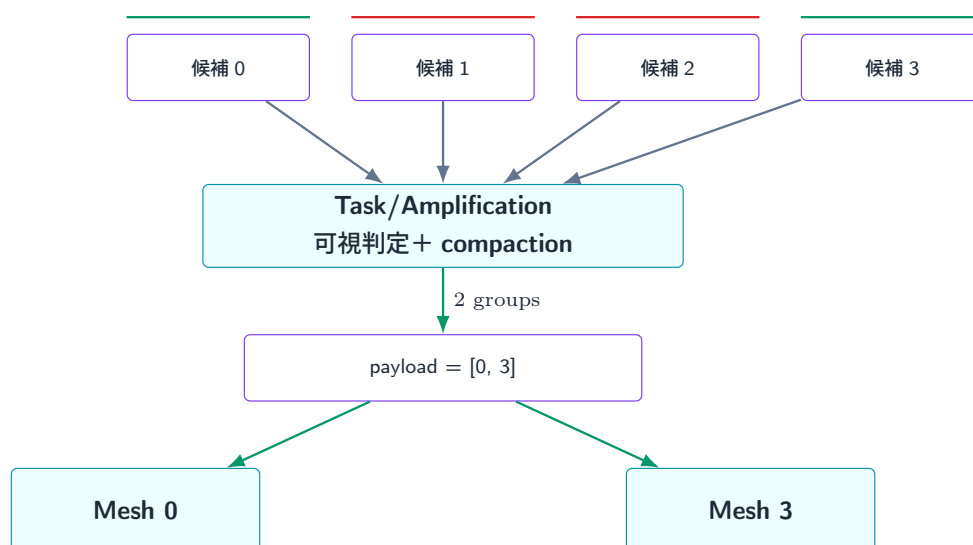


図 8.1: 不可視候補を圧縮して、必要な Mesh Shader だけを発行する

8.2 GLSL Task Shader の基本形

次は 1 Task ワークグループが 32 メッシュレット候補を扱う簡略例である。可視判定結果を atomic 加算で compaction し、全 lane が barrier へ到達した後、uniform に `EmitMeshTasksEXT` を呼ぶ。この関数は Task ワークグループの各 invocation から uniform control flow でちょうど 1 回呼ぶ必要が

ある [5]。

```
1  #version 460
2  #extension GL_EXT_mesh_shader : require
3
4  layout(local_size_x = 32) in;
5
6  struct Payload {
7      uint meshletIds[32];
8  };
9  taskPayloadSharedEXT Payload payload;
10 shared uint visibleCount;
11
12 struct Bounds {
13     vec4 centerRadius;
14 };
15 layout(set = 0, binding = 5, std430) readonly buffer BoundsBuffer {
16     Bounds bounds[];
17 };
18 layout(set = 0, binding = 6) uniform CullData {
19     vec4 frustumPlanes[6]; // xyz = inward normal, w = d
20     uint meshletCount;
21 };
22
23 bool sphereInFrustum(vec3 center, float radius) {
24     for (uint i = 0; i < 6; ++i) {
25         float distance =
26             dot(frustumPlanes[i].xyz, center) +
27             frustumPlanes[i].w;
28         if (distance < -radius) {
29             return false;
30         }
31     }
32     return true;
33 }
34
35 void main() {
36     uint lane = gl_LocalInvocationIndex;
37     if (lane == 0) {
38         visibleCount = 0;
39     }
40     barrier();
41 }
```

```

42     uint candidate = gl_WorkGroupID.x * 32u + lane;
43     bool visible = false;
44     if (candidate < meshletCount) {
45         vec4 sphere = bounds[candidate].centerRadius;
46         visible = sphereInFrustum(sphere.xyz, sphere.w);
47     }
48
49     if (visible) {
50         uint slot = atomicAdd(visibleCount, 1u);
51         payload.meshletIds[slot] = candidate;
52     }
53     barrier();
54
55     EmitMeshTasksEXT(visibleCount, 1u, 1u);
56 }

```

Listing 8.1: cull.task: 32 候補を可視 ID へ圧縮する

対応する Mesh Shader は、親 Task Shader の payload から本当のメッシュレット ID を読む。

```

1 struct Payload {
2     uint meshletIds[32];
3 };
4 taskPayloadSharedEXT Payload payload;
5
6 void main() {
7     uint meshletId = payload.meshletIds[gl_WorkGroupID.x];
8     // 以降は第章の 7Mesh と同じ Shader
9 }

```

Listing 8.2: Task payload を受け取る Mesh Shader 側の変更

Task Shader の API 側グループ数は $\lceil N/32 \rceil$ である。

```

1 const std::uint32_t taskGroups = (meshletCount + 31u) / 32u;
2 cmdDrawMeshTasks(commandBuffer, taskGroups, 1, 1);

```

8.3 HLSL Amplification Shader

HLSL でも考え方は同じである。DispatchMesh は全グループで uniform に 1 回呼び、暗黙に group sync を行って Amplification Shader を終了する [1]。

```
1 struct Payload {
2     uint meshletIds[32];
3 };
4
5 struct Bounds {
6     float4 centerRadius;
7 };
8 StructuredBuffer<Bounds> BoundsBuffer : register(t5);
9
10 cbuffer CullData : register(b1) {
11     float4 FrustumPlanes[6];
12     uint MeshletCount;
13 };
14
15 groupshared Payload gPayload;
16 groupshared uint gVisibleCount;
17
18 bool SphereInFrustum(float3 c, float r) {
19     [unroll]
20     for (uint i = 0; i < 6; ++i) {
21         if (dot(FrustumPlanes[i].xyz, c) +
22             FrustumPlanes[i].w < -r) {
23             return false;
24         }
25     }
26     return true;
27 }
28
29 [numthreads(32, 1, 1)]
30 void ASMain(uint lane : SV_GroupIndex,
31             uint3 groupId : SV_GroupID) {
32     if (lane == 0) gVisibleCount = 0;
33     GroupMemoryBarrierWithGroupSync();
34
35     uint candidate = groupId.x * 32 + lane;
36     bool visible = false;
37     if (candidate < MeshletCount) {
38         float4 s = BoundsBuffer[candidate].centerRadius;
39         visible = SphereInFrustum(s.xyz, s.w);
40     }
41
42     if (visible) {
```



```

43     uint slot;
44     InterlockedAdd(gVisibleCount, 1, slot);
45     gPayload.meshletIds[slot] = candidate;
46 }
47 GroupMemoryBarrierWithGroupSync();
48
49 DispatchMesh(gVisibleCount, 1, 1, gPayload);
50 }

```

Listing 8.3: 32 候補を圧縮する Amplification Shader の概念例

Mesh Shader 関数へ payload 引数を追加し、group ID で可視 ID を引く。

```

in payload Payload payload
// uint meshletId = payload.meshletIds[groupId.x];

```

8.4 atomic compaction から subgroup compaction へ

学習用 atomic 方式は分かりやすいが、全可視 lane が 1 個の counter を更新する。subgroup ballot と prefix count を使えば、各 lane が「自分より前の可視 lane 数」を求め、subgroup ごとの base だけを atomic で確保できる。

$$\text{slot}_{\text{lane}} = \text{base}_{\text{subgroup}} + \text{prefixVisible}_{\text{lane}}. \quad (8.1)$$

ただし、subgroup 数が複数のワークグループでは、subgroup 間の base 計算と barrier が必要になる。最初は atomic 版で正しさを確立し、カリングが本当にボトルネックになった時に置き換える。32 候補程度なら単純版が十分な GPU も多い。

8.5 法線コーンを追加する

視錐台を通過した候補だけに法線コーン判定を行う。安い判定を先に置くことで、不要な内積や正規化を減らす。

```

1 bool coneBackFacing(vec3 center,
2                     vec3 coneAxis,
3                     float cutoff,
4                     vec3 cameraPosition) {
5     vec3 toCamera = normalize(cameraPosition - center);
6     // の定義と符号は生成側と一致させる cutofffasset

```

```

7     return dot(coneAxis, toCamera) <= cutoff;
8 }
9
10 visible = sphereInFrustum(center, radius);
11 if (visible && coneIsValid) {
12     visible = !coneBackFacing(center, axis, cutoff,
13                               cameraPosition);
14 }

```

Listing 8.4: 法線コーン判定を追加する概念コード

この式はデータ定義を説明するための骨格であり、境界球の角度広がりやカメラが球内にいる場合を保守的に扱う必要がある。両面材質には適用しない。

8.6 Task/Amplification Shader を使わない方がよい場合

- ほぼすべてのメッシュレットが可視で、前段が追加仕事になる。
- 1 Task グループが 1 Mesh グループしか発行せず、compaction 効果がない。
- payload が大きく、共有メモリと occupancy を圧迫する。
- 境界判定が Mesh Shader 内の早期 0 出力で十分に軽い。
- GPU profiler で前段の時間が減った Mesh 処理時間を上回る。

AMD 資料は、Task/Amplification 段で多くのメッシュレットをまとめて扱い、payload を小さくすることを推奨している。NVIDIA 資料も、前段が有意な仕事の削減や増幅を行う場合に使う考え方を示している [15, 17]。

演習 8.1: compaction を手計算する

8 lane の可視フラグが [1, 0, 1, 1, 0, 0, 1, 0] である。各 lane が exclusive prefix sum で得る値と、可視 lane が書き込む slot を求めよ。最終 visible count はいくつか。

演習 8.2: barrier を外すと何が起きるか

visibleCount=0 の直後の barrier、atomic 後の barrier をそれぞれ外した場合に起こり得る競合を説明せよ。「CPU では順番に見える」という説明が GPU で通用しない理由も書け。

LOD と GPU 駆動描画

9.1 LOD は三角形数だけの問題ではない

Level of Detail は、遠くの物体を粗くして処理量を減らす仕組みである。メッシュシェーダーでは、モデル全体を 1 段階で切り替えるだけでなく、空間クラスタごとに異なる LOD を選べる。近い建物の正面は細かく、遠い背面は粗くする、といった選択が可能になる。

各クラスタ node に、境界球、幾何誤差、子 node 範囲、leaf ならメッシュレット範囲を持たせる。

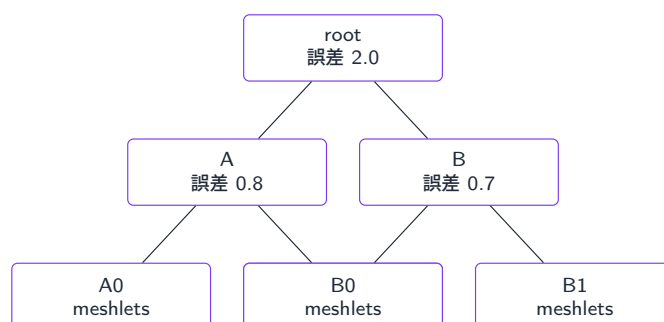


図 9.1: クラスタ階層 LOD の概念。画面誤差が大きい node だけ子へ降りる

9.2 画面誤差で選ぶ

第 4 章の式を使い、node の投影誤差 e_{px} を計算する。

$$e_{px} \approx \frac{e}{z} \frac{H}{2 \tan(f_y/2)}. \quad (9.1)$$

- $e_{px} \leq \tau$: 現在 node の粗い表現を描く。
- $e_{px} > \tau$: 子 node を評価する。
- 視錐台外: node と全子孫を捨てる。

親を描くか子を描くかのどちらか一方にする。両方を描くと重複し、どちらも描かないと穴が開く。並列走査では、この**排他的選択**を保証するデータ構造とアルゴリズムが必要である。

9.3 3つの実装段階

表 9.1: GPU 駆動化の段階

段階	方法	特徴
1: CPU 選択	CPU が LOD と可視物体を選び直接 dispatch	最も簡単、読み戻し不要。CPU 負荷が増える
2: Task 選択	Task Shader が候補と LOD を選び Mesh を発行	pipeline 内で完結。階層走査の形に制約
3: Compute queue	Compute Shader が可視リストと indirect 引数を生成	柔軟。同期と indirect buffer 管理が必要

最初は段階 1 で LOD 式を検証し、次に flat なメッシュレット列を Task Shader で選ぶ。階層の動的 queue、occlusion、streaming を一度に導入しない。

9.4 indirect dispatch

Vulkan は `vkCmdDrawMeshTasksIndirectEXT` と count 版、Direct3D 12 は `ExecuteIndirect` の Mesh Dispatch argument を利用できる [4, 1]。Compute Shader が可視メッシュレット数と dispatch 引数を書き、graphics pass がそれを読む。

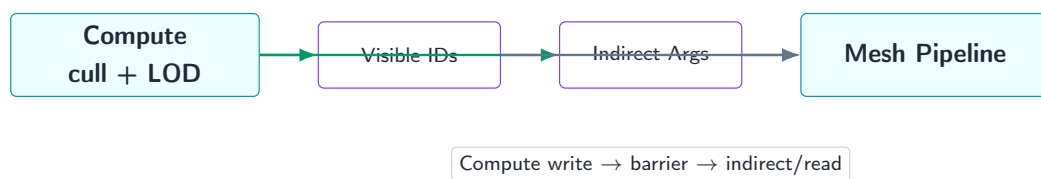


図 9.2: Compute pass で可視リストと indirect 引数を作る構成

この構成では同期が正しさの一部になる。Vulkan なら Compute Shader の storage write を、draw indirect の command read と Mesh Shader の storage read へ可視化する。D3D12 なら UAV 書き込み完了を保証し、argument resource を間接引数として読める状態へ遷移する。詳細は第 13 章で扱う。

9.5 occlusion culling の注意

前フレームの depth pyramid を使う occlusion culling は大きな削減を生むが、誤って見える物を捨てると破綻する。境界球または AABB を画面へ投影し、保守的な矩形と最も近い深度を使う。次の対策を組み合わせる。

- カメラ移動量に応じて判定を緩める。
- 直前まで可視だった物は数フレーム残す。
- depth pyramid の mip 選択を保守的にする。
- near plane を跨ぐ bounds は可視扱いにする。
- デバッグ表示で「視錐台」「法線」「occlusion」「LOD」の破棄理由を色分けする。

9.6 LOD の継ぎ目

隣接クラスタが異なる LOD を選ぶと、境界頂点が一致せず亀裂が生じる。解決策には、境界を固定する、skirt を追加する、親子で境界頂点を共有する、遷移メッシュを用意する、cluster hierarchy 生成時に連続性を制約する方法がある。どれもメモリ・生成時間・見た目のトレードオフである。

ハンズオン: LOD 誤差を色で見る

画面誤差 e_{px} を 0 から 4px で色へ変換し、選択 LOD の上に重ねる。閾値を 0.5、1.0、2.0px へ変え、三角形数、可視メッシュレット数、GPU 時間、見た目を記録する。解像度を 1080p から 4K へ変えたとき、同じ距離閾値方式と画面誤差方式を比較する。

演習 9.1: LOD 閾値を比較する

同じクラスタが 1080p で 0.8px、4K で 1.6px の誤差に見えるとする。閾値 1.0px なら各解像度でどの LOD を選ぶか。距離だけの閾値ではこの差を表せない理由を書け。

演習 9.2: 階層選択の不変条件

親 node と 2 個の子 node がある。親を描く条件、子へ降りる条件、視錐台外条件を擬似コードにし、親と子が同時に visible list へ入らないことを説明せよ。

第Ⅲ部

実用データとレンダラへ育てる

第 10

アセット前処理パイプライン

10.1 ランタイムより先にデータを設計する

メッシュシェーダー導入で最も大きく変わるのは、シェーダーよりアセット変換である。入力モデルをそのまま GPU へ渡すのではなく、頂点を整理し、メッシュレットへ分割し、境界情報と LOD を生成し、GPU が連続読み出しできる形へ並べる。

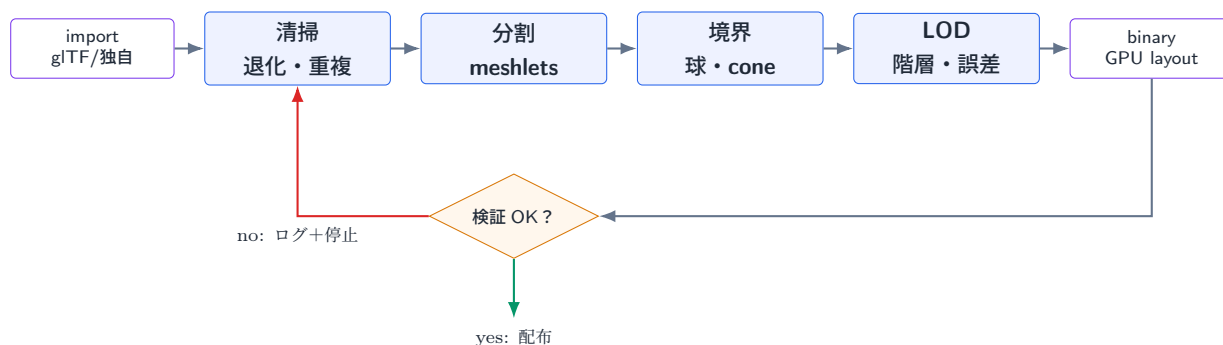


図 10.1: オフラインアセット変換の推奨工程

変換器は「高速であること」より「再現可能で、壊れた入力を説明できること」が重要である。同じ入力・設定・変換器 version から同じ binary が生成されるようにし、設定値と統計を manifest へ残す。

10.2 入力メッシュを清掃する

メッシュレット分割前に次を処理する。

1. index が頂点範囲内か確認する。
2. 面積が閾値以下の退化三角形を除去する。
3. NaN/Inf を含む位置・法線・UV を検出する。
4. 必要に応じて同一属性の重複頂点を結合する。

5. tangent を再生成し、handedness を含める。
6. マテリアル境界、hard normal seam、UV seam を保持する。
7. 座標単位、up axis、handedness をプロジェクト規約へ変換する。

同じ位置でも法線や UV が異なる頂点は、GPU 上では別頂点である。見かけ上の幾何共有だけで結合すると、UV seam が崩れたり、hard edge が滑らかになったりする。頂点同一性は、位置だけでなくシェーダーが読む全属性で決める。

```

1 bool isDegenerate(const Vec3& p0,
2                   const Vec3& p1,
3                   const Vec3& p2,
4                   float epsilonSquared) {
5     const Vec3 n = cross(p1 - p0, p2 - p0);
6     return dot(n, n) <= epsilonSquared;
7 }

```

Listing 10.1: 退化三角形を面積で判定する

閾値はモデル単位に依存する。絶対値だけでなく、モデルの AABB 対角長に対する相対値を使うと、巨大建築と小物の両方へ適用しやすい。

10.3 隣接情報を作る

貪欲分割では、現在メッシュレットに接する三角形を効率よく探するため、edge から triangle への隣接表を作る。無向 edge は $(\min(i, j), \max(i, j))$ を key にする。非多様体 edge には 3 枚以上の三角形が接するため、固定 2 要素ではなく可変リストまたは警告を使う。

```

1 std::uint64_t edgeKey(std::uint32_t a, std::uint32_t b) {
2     const std::uint32_t lo = std::min(a, b);
3     const std::uint32_t hi = std::max(a, b);
4     return (std::uint64_t{hi} << 32u) | lo;
5 }

```

Listing 10.2: 無向 edge key の一例

非多様体や穴があっても描画自体は可能だが、法線コーン、LOD 簡略化、境界の連続性で特別扱いが必要になる。変換器は修復を黙って行わず、元の mesh 名、primitive 番号、triangle 番号をログに含める。

10.4 メッシュレットを生成する

次の擬似コードは、第 3 章の貪欲法を少し具体化したものである。

```

1  while (existsUnassignedTriangle()) {
2      MeshletBuild current;
3      current.add(pickSeedTriangle());
4
5      CandidateQueue queue;
6      queue.pushAdjacent(current);
7
8      while (!queue.empty()) {
9          TriangleId t = queue.popLowestScore();
10         if (isAssigned(t)) continue;
11
12         const auto delta = evaluateAddition(current, t);
13         if (current.vertexCount + delta.newVertices > maxVertices)
14             continue;
15         if (current.primitiveCount + 1 > maxPrimitives)
16             continue;
17
18         current.add(t);
19         queue.pushNewAdjacent(t);
20     }
21
22     finalize(current);
23 }
```

Listing 10.3: 学習用のメッシュレット生成擬似コード

seed 選択は結果に影響する。未処理 triangle の先頭だけでなく、孤立領域、曲率が高い領域、境界 triangle を優先する方式を比較できる。生成結果を安定させるため、score 同値時の tie-break を triangle ID など固定する。

10.5 境界球を生成する

最も単純には、メッシュレット頂点の AABB 中心を球中心とし、そこから最遠頂点までを半径にする。

$$\mathbf{c} = \frac{\mathbf{b}_{min} + \mathbf{b}_{max}}{2}, \quad r = \max_i \|\mathbf{p}_i - \mathbf{c}\|. \quad (10.1)$$

これは最小球ではないが、必ず全頂点を含み、実装しやすい。より小さな球が必要なら Ritter 法などを使い、最後に全頂点を再走査して丸め誤差を含めても外へ出ないように半径を拡張する。

```

1  for (std::uint32_t v : meshlet.globalVertexIndices) {
2      const float distance = length(positions[v] - center);
3      if (distance > radius + validationEpsilon) {
4          reportError(meshletId, v, distance, radius);
5      }
6  }
```

Listing 10.4: 境界球を検証する

10.6 法線コーンを生成する

学習用の保守的な生成法は次の通りである。

1. 各非退化 triangle の単位法線 $\mathbf{n}_i$ を求める。
2. 面積で重み付けした法線和を正規化し、軸 $\mathbf{a}$ とする。
3. $m = \min_i(\mathbf{a} \cdot \mathbf{n}_i)$ を求める。
4. m が負に近い、または法線和がほぼ 0 なら cone を無効にする。
5. cutoff の定義と符号を binary format へ明記する。

$$\mathbf{a} = \frac{\sum_i A_i \mathbf{n}_i}{\|\sum_i A_i \mathbf{n}_i\|}, \quad m = \min_i(\mathbf{a} \cdot \mathbf{n}_i). \quad (10.2)$$

この方法は厳密な最小 cone ではないが、デバッグしやすい。全法線が cone 内に入ることを変換器で検証し、量子化後の axis/cutoff でも再検証する。

10.7 LOD と誤差を生成する

簡略化前の頂点と簡略化後の surface 間の最大距離を、幾何誤差の候補とする。厳密計算が高価なら、元頂点から簡略化 surface への距離をサンプリングして上限を安全側へ膨らませる。重要なのは、誤差が「小さいほど高品質」という単調性を親子階層で保つことである。

$$e_{parent} \geq \max(e_{child_0}, e_{child_1}, \dots). \quad (10.3)$$

親の誤差が子より小さいと、画面誤差判定が距離に対して逆転し、LOD が不安定になる。生成後に

階層を bottom-up で走査して修正できる。

10.8 binary format を version 化する

表 10.1: メッシュアセット header に含めたい項目

項目	目的
magic / formatVersion	別ファイルや古い layout の誤読を防ぐ
endianness / flags	platform 差、圧縮方式、index 幅を示す
converterVersion	再現性と不具合追跡
sourceHash / setting-sHash	入力・変換設定の一致確認
各 section offset/size/-count	bounds check と memory map
最大頂点数 / 最大 primitive 数	shader profile との一致確認
AABB / sphere	scene level culling と sanity check
CRC または content hash	破損検出

offset 加算は整数 overflow を検査する。 $offset + size \leq fileSize$ を直接計算すると overflow する可能性があるため、 $offset \leq fileSize$ かつ $size \leq fileSize - offset$ の順に確認する。外部から取得したアセットを読む場合、GPU へ渡す前の bounds check はセキュリティ上も重要である。

10.9 統計と可視化

変換器は次を CSV または JSON へ出力する。

- meshlet 数、平均・最小・最大頂点数/三角形数
- 頂点充填率、primitive 充填率、頂点再利用指標
- 境界球半径の分布、法線 cone 角の分布
- cone 無効 meshlet 比率、退化 triangle 数
- material ごとの meshlet 数と小さすぎる meshlet 数

- 各 LOD の三角形数、誤差、親子数

平均だけでは、1 三角形 meshlet が大量に混じる問題を隠す。percentile や histogram を出し、最悪例の mesh/meshlet ID をビューアで開けるようにする。

ハンズオン: 変換器の最初の完成条件

立方体、UV 球、平面、細長い帯、非多様体を含む人工モデルを変換する。変換後の全 triangle を CPU で再構成し、入力 triangle と属性込みで一致することを確認する。さらに Mesh Shader 表示と通常 index 表示を重ね、depth 差を色表示する。

演習 10.1: 分割 score を比較する

$S = w_v \Delta V + w_r \Delta R$ とする。候補 A は $\Delta V = 1, \Delta R = 0.4$ 、候補 B は $\Delta V = 2, \Delta R = 0.1$ である。(a) $w_v = 1, w_r = 1$ 、(b) $w_v = 1, w_r = 5$ で選ばれる候補を求め、結果の形状を推測せよ。

演習 10.2: binary validation を設計する

header が示す Meshlet section の offset、element size、count から必要 byte 数を検証する擬似コードを書け。乗算 overflow と加算 overflow の両方を避けること。

第 11

頂点属性・マテリアル・圧縮

11.1 位置だけから実用頂点へ

実用モデルには位置、法線、接線、UV、頂点色、skin weight などがある。Mesh Shader は固定 vertex input layout に縛られないため、Storage/Structured/ByteAddress Buffer から自由な形式を読める。自由度は高いが、decode ALU、メモリ帯域、alignment、shader variant の責任もアプリが負う。

表 11.1: 代表的な属性形式とトレードオフ

属性	学習用	圧縮候補	注意
position	float3/float4	mesh AABB 内 UNORM16x3	大きな world 座標は object-local で量子化
normal	float3	octahedral SNORM16x2	decode 後に正規化
tangent	float4	octahedral + handedness bit	normal との直交化
UV	float2	half2 / UN- ORM16x2	tile 範囲や負値を確認
color	float4	UNORM8x4	sRGB/linear の意味を固定
weights	float4+uint4	UNORM8x4 + uint8x4	和を再正規化、joint 数上限

11.2 量子化位置を復元する

モデルまたは cluster の AABB を $\mathbf{b}_{min}, \mathbf{b}_{max}$ とし、各成分を 16bit unsigned へ量子化する。 $q \in [0, 65535]$ から位置を復元する式は

$$\mathbf{p} = \mathbf{b}_{min} + \frac{\mathbf{q}}{65535} \odot (\mathbf{b}_{max} - \mathbf{b}_{min}), \quad (11.1)$$

ここで $\odot$ は成分ごとの積である。広いモデル全体の AABB を使うと精度が落ちるため、mesh、LOD cluster、streaming page など適切な局所範囲を選ぶ。

最大量子化 step は各軸で

$$\Delta_x = \frac{b_{max,x} - b_{min,x}}{65535} \quad (11.2)$$

である。LOD 誤差より量子化誤差が大きい場合、LOD 評価の意味が崩れる。変換器で量子化前後の最大位置誤差を測り、許容値へ含める。

11.3 octahedral normal encoding

単位法線 $\mathbf{n} = (x, y, z)$ を 2 成分へ写す代表的方法である。まず L1 norm で正規化する。

$$\mathbf{p} = \frac{(x, y)}{|x| + |y| + |z|}. \quad (11.3)$$

$z < 0$ なら正方形の反対側へ fold する。

$$\mathbf{p}' = (1 - |p_y|, 1 - |p_x|) \odot \text{sign}(\mathbf{p}). \quad (11.4)$$

2 成分を SNORM16 などへ量子化する。復元後は必ず正規化する。

```

1 float3 DecodeOctNormal(float2 e) {
2     float3 n = float3(e.x, e.y,
3                       1.0 - abs(e.x) - abs(e.y));
4     if (n.z < 0.0) {
5         n.xy = (1.0 - abs(n.yx)) *
6               float2(n.x >= 0.0 ? 1.0 : -1.0,
7                     n.y >= 0.0 ? 1.0 : -1.0);
8     }
9     return normalize(n);
10 }
```

Listing 11.1: octahedral normal の decode 例

境界値、ゼロ、南極付近を CPU/GPU で同じ入力に通し、最大角度誤差を測る。圧縮は画像が出た後に行い、圧縮前後をキー 1 つで切り替えられるようにする。

11.4 AoS と SoA

Array of Structures は 1 頂点の属性をまとめる。**Structure of Arrays** は position 配列、normal 配列、UV 配列のように分ける。

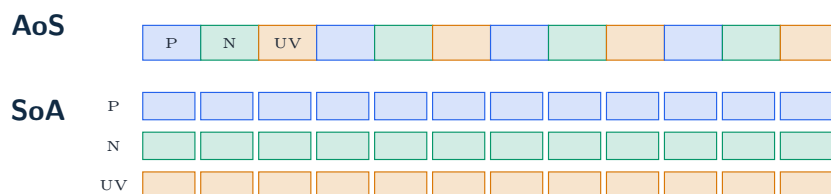


図 11.1: AoS と SoA の概念。必要属性だけ連続読み出しできるかが違う

Depth-only pass では position しか使わないため、SoA なら normal や UV を読まずに済む。一方、full material pass で全属性を使い、vertex 単位でまとまった cache line が有効なら AoS が扱いやすい。中間案として、position 単独、normal+tangent、UV+color の複数 stream へ分ける。

11.5 per-vertex と per-primitive

Mesh Shader は triangle ごとの属性を直接出力できる。GLSL では `perprimitiveEXT`、HLSL では `out primitives` を使う。flat な material ID や debug meshlet ID を 3 頂点へ重複させずに済む。

```

1 layout(location = 0) out vec3 outNormal[];
2 layout(location = 1) perprimitiveEXT out uint outMaterialId[];
3
4 // を書いている内 primitiveloop
5 outMaterialId[p] = meshletMaterialId;
```

Listing 11.2: GLSL の per-primitive material ID

Fragment Shader 側も対応する入力を flat かつ `perprimitiveEXT` の interface として受ける実装規則を確認する。HLSL では Mesh Shader の primitive output 構造に `nointerpolation uint materialId : MATERIAL0;` を置き、Pixel Shader 入力 semantic を一致させる。

```

1 struct PrimitiveOut {
2     nointerpolation uint materialId : MATERIAL0;
3 };
4
5 [outputtopology("triangle")]
6 [numthreads(32, 1, 1)]
```

```

7 void MSMain(
8     // ...
9     out primitives PrimitiveOut prims[MAX_PRIMS]) {
10    // の後 SetMeshOutputCounts
11    if (lane < primitiveCount) {
12        prims[lane].materialId = materialId;
13    }
14 }

```

Listing 11.3: HLSL の primitive output 例

11.6 material を混ぜるか分けるか

1 メッシュレット 1 マテリアルは、分岐・descriptor 選択・透明度分類を単純にする。ただし、細かな material 境界が多いモデルでは小さなメッシュレットが増える。複数 material を許可する場合は triangle ごとの material ID を持ち、Pixel Shader で texture 配列や bindless descriptor を選ぶ。

表 11.2: マテリアル分割方針

方針	長所	短所
1 meshlet = 1 material	分岐が揃い、pipeline 分類が簡単	小 meshlet 増加、draw 分類数増加
primitive ごと material	充填率が高く、分割自由度が高い	descriptor の非一様アクセス、Pixel 分岐
hybrid	主な material で分け、小領域のみ primitive ID	変換器と shader が複雑

descriptor indexing/bindless を使う場合も、範囲外 material ID を GPU へ渡さない。null/default texture を 0 番に置き、変換時とロード時の両方で clamp ではなく validation する。不正値を黙って 0 へ丸めると、破損原因を隠す。

11.7 透明・両面材質

alpha blend 材質は通常、深度順序の影響を受ける。Task Shader で可視 ID を圧縮しても、順序を保たない atomic compaction では描画順が変わる。weighted blended OIT など順序非依存方式を使うか、透明 meshlet を別 queue へ出し、sort key を生成する。

両面材質では次を変える。

- rasterizer cull mode を none にする。
- 法線 cone による back-face culling を無効にする。
- Pixel Shader で front/back face に応じて normal を反転するか材質規約を決める。
- foliage では alpha test と small triangle の sample coverage を評価する。

演習 11.1: 量子化誤差を求める

X 軸範囲が 100m の model を UNORM16 で量子化する。1 step は何 m、何 mm か。X 軸範囲が 1m の cluster 単位ならどうなるか。

演習 11.2: 属性 stream を設計する

Depth-only、shadow、G-buffer、forward transparent の 4 pass について必要属性を表にせよ。その結果から、position、normal/tangent、UV/color、skin data をどの stream へ分けるか提案せよ。

第 12

アニメーションとプロシージャル生成

12.1 スキニングをどこで行うか

skeletal animation では、joint 行列と weight から頂点位置・法線を変形する。メッシュシェーダー導入時の主な選択は次の 3 つである。

表 12.1: スキニング位置の選択

方法	長所	短所
Mesh Shader 内	可視 meshlet の頂点だけ変形、追加 buffer 不要	pass ごとに再計算、bounds 判定前の姿勢が分からない
Compute で事前変形	depth/shadow/color で結果を再利用、bounds 更新もしやすい	変形頂点 buffer と同期・帯域が必要
Vertex Shader fallback と共有	既存経路を流用しやすい	Mesh 固有の局所再利用を活かさない

Mesh Shader 内で行う場合、同じ全体頂点を複数メッシュレットが参照すると、その頂点のスキニングを複数回計算する。一方、不可視メッシュレットなら計算しない。Compute 事前変形は全可視 instance の全頂点を処理する可能性があるが、複数 pass で再利用できる。scene、pass 数、可視率で測る。

12.2 スキニングの式

4 joint の linear blend skinning は

$$\mathbf{p}' = \sum_{i=0}^3 w_i B_i \mathbf{p}, \quad \sum_i w_i = 1 \quad (12.1)$$

である。 B_i は bind pose 逆行列と現在 joint 行列を組み合わせた skin matrix である。weight 量子

化後は和が 1 にならないことがあるため、decode 後に再正規化する。和が 0 なら joint 0 を weight 1 にするなど、明示的な fallback を設ける。

法線は位置と同じ $w = 1$ で変換しない。translation を除いた 3x3 部を使い、非一様 scale がある場合は inverse transpose が必要になる。多くの real-time rig が uniform scale を前提にするなら、その制約を asset validator で検査する。

```

1  float4 SkinPosition(float3 p,
2                      uint4 joints,
3                      float4 weights) {
4      float sumW = dot(weights, 1.0.xxxx);
5      weights = (sumW > 1e-6) ? weights / sumW
6                      : float4(1, 0, 0, 0);
7
8      float4 hp = float4(p, 1.0);
9      return mul(SkinMatrices[joints.x], hp) * weights.x +
10             mul(SkinMatrices[joints.y], hp) * weights.y +
11             mul(SkinMatrices[joints.z], hp) * weights.z +
12             mul(SkinMatrices[joints.w], hp) * weights.w;
13 }

```

Listing 12.1: Mesh Shader 内スキニングの簡略例

12.3 静的 bounds は変形後を含むか

bind pose の境界球だけで animated meshlet を cull すると、腕や布が外へ動いたときに見える geometry を捨てる。対策は次の通りである。

1. animation clip 全体を sample し、meshlet ごとの保守的 bounds を offline 生成する。
2. joint ごとの影響 bounds を持ち、現在 pose から GPU/CPU で合成する。
3. Compute skinning 後に各 meshlet bounds を parallel reduction で更新する。
4. bounds 更新が難しい物は object-level culling だけにし、meshlet culling を無効化する。

clip 全体 bounds は安価だが大きくなり、カリング効果が下がる。毎 frame 更新は正確だが計算と同期が増える。character 数と遮蔽率で段階を選ぶ。

12.4 morph target

morph は base position へ複数 delta を加える。

$$\mathbf{p}' = \mathbf{p} + \sum_k \alpha_k \Delta \mathbf{p}_k. \quad (12.2)$$

active weight が少ない場合だけ sparse delta を読む構成が有効である。ただし lane ごとに active target 数が異なると分岐と間接読み出しが増える。顔など重要な mesh は Compute でまとめて変形し、Mesh Shader は結果を読む設計も比較する。

morph でも bounds が変わる。各 target の最大 delta を使って半径を膨らませる簡易方式は安全だが保守的である。複数 target の同時最大が実際には起こらない rig なら、許可される weight 組み合わせを validator へ与える。

12.5 プロシージャル geometry

Mesh Shader は入力 vertex buffer なしでも geometry を生成できる。適用例は次の通りである。

- grass blade、billboard、particle quad
- voxel surface、marching cubes の小 block
- terrain patch の規則格子
- line/curve の ribbon 化
- debug sphere、frustum、bounds 表示

たとえば 1 粒子を 4 頂点・2 三角形の quad へ展開する。32 lane が 32 粒子を担当すると、最大 128 頂点・64 三角形を出す。不可視粒子を prefix compaction してから、4V, 2V を出力数に設定する。

```

1  // visibleCount と slot は group-wide で得る compaction
2  SetMeshOutputCounts(visibleCount * 4,
3                      visibleCount * 2);
4
5  if (isVisible) {
6      uint vertexBase = slot * 4;
7      uint primBase   = slot * 2;
8      // camera right/から up4 を出力 corner
9      // triangles[primBase + 0] = ...
10     // triangles[primBase + 1] = ...
11 }

```

Listing 12.2: 可視 particle を quad へ展開する設計の骨格

出力上限から逆算し、1 グループの最大可視粒子数を決める。256 頂点上限なら quad は最大 64 個だが、出力属性 byte 数やワークグループ threads の上限が先に効くこともある。

12.6 tessellation の置き換え

規則的な patch を Mesh Shader で細分化できるが、固定 function tessellator と同じ仕組みではない。各 patch が必要分割数を決め、格子頂点と triangle を明示的に生成する。隣接 patch の edge 分割数を一致させないと亀裂が生じる。

$$V = (u + 1)(v + 1), \quad P = 2uv \quad (12.3)$$

である。出力上限から u, v を選ぶ。たとえば $u = v = 8$ なら 81 頂点・128 三角形である。曲面評価が重い場合、各頂点を lane で分担し、triangle 接続を別 loop で書く。

演習 12.1: particle 上限を計算する

1 粒子 4 頂点・2 三角形、device 上限が最大 256 頂点・256 三角形とする。出力数だけから決まる 1 グループ最大粒子数はいくつか。頂点属性が 1 頂点 32byte で、Mesh 出力 data 上限 32KiB も考慮すると変わるか、概算せよ。

演習 12.2: animated bounds 方針を選ぶ

100 体の character、3 つの render pass、画面外が 70% という scene を考える。Mesh Shader 内 skin、Compute 事前 skin、offline 保守 bounds の組み合わせを提案し、計算の重複とカリニング精度を論じよ。

第 13

同期・メモリ・リソース寿命

13.1 正しいデータでも同期がなければ壊れる

Compute Shader が visible list と indirect args を書き、その直後に Mesh Shader を起動する構成を考える。同じ command buffer/list 内に命令を順番に記録しただけでは、必要な memory visibility が自動で保証されない場合がある。**実行依存**と**メモリ依存**を明示する。

直感でつかむ

barrier は「GPU 全体を止める呪文」ではない。誰のどの書き込みを、次の誰のどの読み込みから見えるようにするかを指定する契約である。範囲を広げすぎると遅く、狭すぎると壊れる。

13.2 Vulkan Synchronization2 の例

Compute pass が 2 つの buffer へ書く。indirectArgs は draw indirect engine が読み、visibleIds は Mesh Shader が読む。

```
1  VkBufferMemoryBarrier2 barriers[2]={};
2
3  barriers[0].sType =
4      VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2;
5  barriers[0].srcStageMask =
6      VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT;
7  barriers[0].srcAccessMask =
8      VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT;
9  barriers[0].dstStageMask =
10     VK_PIPELINE_STAGE_2_DRAW_INDIRECT_BIT;
11 barriers[0].dstAccessMask =
12     VK_ACCESS_2_INDIRECT_COMMAND_READ_BIT;
13 barriers[0].buffer = indirectArgs;
14 barriers[0].offset = 0;
15 barriers[0].size = VK_WHOLE_SIZE;
16
17 barriers[1].sType =
```

```

18     VK_STRUCTURE_TYPE_BUFFER_MEMORY_BARRIER_2;
19     barriers[1].srcStageMask =
20         VK_PIPELINE_STAGE_2_COMPUTE_SHADER_BIT;
21     barriers[1].srcAccessMask =
22         VK_ACCESS_2_SHADER_STORAGE_WRITE_BIT;
23     barriers[1].dstStageMask =
24         VK_PIPELINE_STAGE_2_MESH_SHADER_BIT_EXT;
25     barriers[1].dstAccessMask =
26         VK_ACCESS_2_SHADER_STORAGE_READ_BIT;
27     barriers[1].buffer = visibleIds;
28     barriers[1].offset = 0;
29     barriers[1].size = VK_WHOLE_SIZE;
30
31     VkDependencyInfo dependency{
32         VK_STRUCTURE_TYPE_DEPENDENCY_INFO
33     };
34     dependency.bufferMemoryBarrierCount = 2;
35     dependency.pBufferMemoryBarriers = barriers;
36
37     vkCmdPipelineBarrier2(commandBuffer, &dependency);

```

Listing 13.1: Compute write から indirect/Mesh read への Vulkan barrier

同じ buffer に argument と ID を混在させる場合は、dst stage/access を OR し、正しい byte range を指定する。queue family をまたぐなら ownership transfer も必要である。1 つの graphics queue で Compute と Mesh を順に使う最初の実装では、queue family は ignored のままにできる。

13.3 upload から Mesh Shader read へ

staging buffer から device local buffer へ copy した後、Mesh Shader が storage buffer として読む。

```

1     barrier.srcStageMask = VK_PIPELINE_STAGE_2_COPY_BIT;
2     barrier.srcAccessMask = VK_ACCESS_2_TRANSFER_WRITE_BIT;
3     barrier.dstStageMask = VK_PIPELINE_STAGE_2_MESH_SHADER_BIT_EXT |
4                             VK_PIPELINE_STAGE_2_TASK_SHADER_BIT_EXT;
5     barrier.dstAccessMask = VK_ACCESS_2_SHADER_STORAGE_READ_BIT;

```

Listing 13.2: Transfer write から Mesh Shader read への要点

Task Shader を使わない場合、対応 stage bit を入れる必要はない。同じ asset を Vertex Shader fallback でも読むなら、VK_PIPELINE_STAGE_2_VERTEX_SHADER_BIT や vertex/index input 用 ac-

cess を実際の用途に合わせて追加する。「将来使うかもしれない全 stage」を入れるより、resource usage をコードで追跡できる仕組みを作る。

13.4 D3D12 の barrier

Compute が UAV として書いた後、同じ resource を indirect argument として使う場合、UAV 処理の順序と resource state transition を行う。従来 barrier API の概念例は次の通りである。

```
1 D3D12_RESOURCE_BARRIER uav{};
2 uav.Type = D3D12_RESOURCE_BARRIER_TYPE_UAV;
3 uav.UAV.pResource = indirectArgs.Get();
4 commandList->ResourceBarrier(1, &uav);
5
6 D3D12_RESOURCE_BARRIER transition{};
7 transition.Type = D3D12_RESOURCE_BARRIER_TYPE_TRANSITION;
8 transition.Transition.pResource = indirectArgs.Get();
9 transition.Transition.StateBefore =
10     D3D12_RESOURCE_STATE_UNORDERED_ACCESS;
11 transition.Transition.StateAfter =
12     D3D12_RESOURCE_STATE_INDIRECT_ARGUMENT;
13 transition.Transition.Subresource =
14     D3D12_RESOURCE_BARRIER_ALL_SUBRESOURCES;
15 commandList->ResourceBarrier(1, &transition);
```

Listing 13.3: D3D12 で UAV から INDIRECT_ARGUMENT へ遷移する概念例

visible ID buffer を Mesh Shader の SRV として読む場合は、resource creation と state strategy に応じて NON_PIXEL_SHADER_RESOURCE へ遷移する。同一 resource へ UAV write を続ける frame では、状態を戻す。Enhanced Barriers を採用する場合も、producer/consumer の考え方は同じである。

13.5 frame 間の競合を避ける

CPU が frame $n + 1$ の constant や visible count を上書きしている間に、GPU が frame n を読んでると競合する。frames-in-flight 数だけ ring を分ける。



図 13.1: frame ごとに書き換え buffer 領域を分離する ring 構成

static meshlet data は全 frame で共有し、camera constants、visible list、indirect args、statistics counter など書き換える resource だけを ring 化する。最大可視数を超えた場合の overflow flag を用意し、debug build では描画停止と明確なログを出す。release build では安全に clamp しても、telemetry へ記録する。

13.6 descriptor 更新と寿命

GPU が参照中の descriptor を CPU が別 resource へ差し替えると、古い command が新しい resource を見る危険がある。descriptor set/heap も frame ring または lifetime tracking を使う。Bindless index を再利用する場合、全 in-flight frame が完了してから free list へ戻す。

asset streaming では次の順序を守る。

1. 新 resource を作り upload する。
2. upload 完了を GPU timeline/fence で確認可能にする。
3. 将来 frame の descriptor/table へ公開する。
4. 古い resource を参照する最後の frame fence 完了後に破棄する。

13.7 statistics counter の読み戻し

可視 meshlet 数や出力 primitive 数を GPU atomic counter へ書くと学習に役立つ。毎 frame 同期して CPU へ読むと stall するため、数 frame 遅れの readback ring を使う。

$$\text{displayFrame} = \text{currentFrame} - \text{readbackLatency}. \quad (13.1)$$

画面表示に「3 frames delayed」などと明記する。性能測定では timestamp query を使い、CPU wall clock と混同しない。

演習 13.1: barrier の stage/access を選ぶ

Compute Shader が buffer A へ visible ID、buffer B へ dispatch args を書き、その後 Mesh Shader が A を、indirect engine が B を読む。Vulkan Synchronization2 の src/dst stage と access を表にせよ。

演習 13.2: ring buffer 容量を計算する

frames-in-flight=3、各 frame の visible ID 最大 200,000 個、1 ID 4byte、indirect args 16byte、statistics 256byte とする。256byte alignment で各 section を配置したとき、1 frame と全 ring のおよその byte 数を求めよ。

第Ⅳ部

性能・品質・製品化

NVIDIA と Radeon を横断して最適化する

14.1 最初に測定条件を固定する

GPU 最適化で最も危険なのは、1 回だけ測った数字を結論にすることである。shader compile、pipeline cache、asset streaming、OS background task、GPU clock 変動を分離する。

チェックリスト

- 同じ camera path、同じ解像度、同じ render settings を使う。
- 数十 frame を warm-up し、その後数百 frame を測る。
- 平均だけでなく median、p95、最大値を記録する。
- CPU frame time と GPU timestamp 区間を分ける。
- VSync、frame limiter、dynamic resolution の状態を記録する。
- GPU、driver、OS、API、build commit、shader hash を保存する。
- NVIDIA と Radeon の両方で同じ実験表を埋める。

性能値は「製品 A が常に製品 B より速い」という判定ではなく、**どの設計が複数の architecture で安定するか**を見るために使う。GPU 名を条件分岐に使う前に、workgroup size、meshlet profile、Task 有無を runtime option として計測する。

14.2 ボトルネックを段階分解する

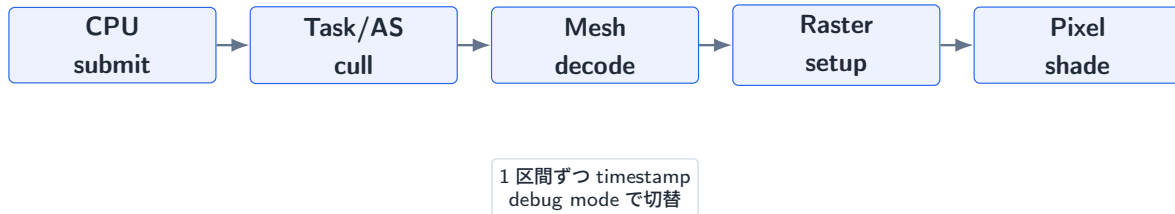


図 14.1: 性能を pipeline 区間へ分けて考える

Pixel Shader が支配的な scene で meshlet size を変えても、frame time はほとんど動かない。まず depth-only または simple color pass で geometry 側を測り、次に本来の material へ戻す。解像度を半分にして速くなるなら pixel 側、Mesh Shader を固定の小 triangle へ置き換えて速くなるなら geometry/data 側が疑わしい。

14.3 meshlet profile を sweep する

最低でも次の組み合わせを offline 生成し、同じ scene で比較する。

表 14.1: 推奨する profile sweep 例

名称	最大頂点	最大三角形	threads	Task 候補数	備考
P0	32	32	32	32	小粒度、culling 重視
P1	64	64	32	32	基準
P2	64	126	32/64	32/64	NVIDIA 資料で代表的
P3	128	128	64	64	頂点多め
P4	128	256	64/128	64/128	AMD 資料で比較候補

NVIDIA は Turing 世代の導入資料で 64 頂点・126 primitive を推奨例として示している。AMD の最適化資料は 128 頂点・256 primitive も有力候補として示しつつ、64/126 も近い性能になる場合があると説明している。どちらも絶対規則ではなく、scene と shader output の大きさで変わる [16, 15]。

各 profile について次を記録する。

$$U_v = \frac{\sum \text{actual vertices}}{\text{meshlet count} \times \text{declared max vertices}}, \quad (14.1)$$

$$U_p = \frac{\sum \text{actual primitives}}{\text{meshlet count} \times \text{declared max primitives}}. \quad (14.2)$$

充填率が高くても境界球が大きくなり、不可視 triangle を余分に描くことがある。[出力利用率](#)、[可視率](#)、[vertex duplication](#)、[GPU 時間](#)を同時に見る。

14.4 出力宣言を小さくする

Mesh Shader の compile-time 最大出力は、実際の出力が少なくても hardware resource allocation へ影響し得る。同じ asset が最大 40 頂点なら、64 用 shader を使い、128/256 用を流用しない。複数 profile ごとに shader variant を用意し、asset header の profile ID で選ぶ。

表 14.2: shader variant を増やす判断

増やす価値が高い	増やしすぎの兆候
出力最大値が大きく異なる asset 群	pipeline cache 作成が長い
skin 有無で resource/ALU が大差	build 時間と配布 binary が膨らむ
depth-only と full material 片面/両面で culling 方針が異なる	runtime で variant 選択ミスが増える ほとんど使われない組合せが多い

variant key を bit field で管理し、実際に使用された key を telemetry で集める。利用されない組み合わせは削除する。

14.5 workgroup size と subgroup

32、64、128 threads を比較する。NVIDIA の warp は一般に 32 lane、AMD の wave は実行 mode により 32 または 64 lane になり得るため、「メーカーごとに固定」ではなく、shader compiler と device subgroup property を確認する。

threads を増やすと、126 triangle の書き込み反復は減るが、64 頂点の meshlet では余る lane が

増える。さらに register と shared memory usage が増えると occupancy が下がる。次の式は単純な lane 利用の目安である。

$$E_v = \frac{V}{\lceil V/T \rceil T}, \quad E_p = \frac{P}{\lceil P/T \rceil T}, \quad (14.3)$$

T は threads、 V, P は実際の頂点・primitive 数である。ただし、vertex loop と primitive loop が重なる実行、memory latency hiding、subgroup scheduling はこの式だけでは分からない。

14.6 i 番 lane が i 番 output を書く

連続 lane が連続 address へアクセスすると、coalescing しやすい。基本形は

```
1 for (uint i = lane; i < count; i += WORKGROUP_SIZE) {
2     output[i] = process(input[i]);
3 }
```

である。仕事量が頂点ごとに大きく異なる場合は load balance が必要だが、まず規則的な mapping を使う。AMD の資料は、local invocation index と出力 index を対応させる構成を推奨している [15]。

global vertex reference がランダムだと、position/attribute 読み込みが散らばる。meshlet 生成時に、同一 meshlet 内の global vertex を近い順に並べ、vertex buffer 自体を再 order する。通常 index fallback も同じ order へ更新する。

14.7 payload と共有メモリを小さくする

Task/Amplification payload には、通常は可視 meshlet ID と少数の共通値だけを入れる。vertex data や大きな matrix 配列を入れず、Mesh Shader が buffer から読む。

表 14.3: payload へ入れる判断

候補	入れる理由	入れない理由
meshlet ID 列	compaction 結果そのもの	ほぼ必須
instance ID	child が同じ instance を参照	dispatch/group から復元可能なら不要
LOD level	debug/offset 計算に使う	meshlet descriptor にあるなら重複
view-projection 行列	なし	大きく、uniform buffer から読める
vertex data	なし	payload を圧迫し再利用しにくい

D3D12 仕様上 payload は最大 16KiB だが、上限まで使うことは目標ではない。Mesh Shader output と groupshared の合計資源が occupancy へ影響する。公式資料でも payload を小さく保つ方針が共通している [1, 17, 15]。

14.8 Task 段の採算を数える

Task/AS の効果を、次の概算で考える。

$$C_{with} = C_{task} + N_{visible}C_{mesh}, \quad C_{without} = N_{candidate}(C_{test} + C_{mesh0}), \quad (14.4)$$

C_{mesh0} は 0 出力 Mesh Shader の費用である。可視率が高い、Task 判定が重い、1 Task group あたり候補が少ない場合は C_{with} が大きくなる。Task on/off を runtime option にし、camera path ごとに損益分岐を測る。

14.9 raster と small triangle

メッシュレット処理が速くても、遠景で 1 pixel 未満の triangle を大量に出すと rasterizer が詰まる。LOD を粗くする、small primitive culling を追加する、cluster を画面誤差で選ぶ。wireframe screenshot だけで品質を判断せず、実解像度の shaded image と temporal stability を見る。

法線 cone culling は triangle を丸ごと削減するが、両面材質や大きな cone では効かない。scene 統計に破棄理由ごとの counter を持つ。

14.10 メーカー固有最適化を隔離する

device property や benchmark 結果から profile を選ぶのはよいが、core shader logic を if (vendor == ...) だらけにしない。

```

1 struct MeshTuningProfile {
2     std::uint32_t maxVertices;
3     std::uint32_t maxPrimitives;
4     std::uint32_t meshThreads;
5     std::uint32_t taskCandidates;
6     bool useTaskShader;
7 };
8
9 MeshTuningProfile selectProfile(
10     const DeviceCaps& caps,
```



```
11     const BenchmarkDatabase& measured,  
12     SceneClass sceneClass);
```

Listing 14.1: profile 選択を data-driven にする例

初回起動の短い microbenchmark、開発時の機種別 database、保守的 default を組み合わせる。未知 GPU では API 上限を満たす P1 などへ戻す。

演習 14.1: lane 効率を比較する

実際の頂点数 50、primitive 数 100 について、threads 32 と 64 の E_v, E_p を計算せよ。この値だけで速い方を断定できない理由を 3 つ挙げよ。

演習 14.2: 実験表を作る

P1、P2、P4 の 3profile、Task on/off、NVIDIA/Radeon 各 1 台を比較する実験表を設計せよ。GPU 時間以外に最低 6 個の列を含めること。

デバッグとプロファイリング

15.1 デバッグ層を常時使える構成にする

Vulkan では validation layer、debug utils、synchronization validation を開発 build で有効化する。D3D12 では debug layer、GPU-based validation、DRED を利用できる構成にする。検証を有効にすると速度は落ちるため、性能計測 build とは分けるが、同じ scene と操作で再現できるようにする。

表 15.1: 不具合の症状から最初に見る場所

症状	よくある原因	最初の確認
何も出ない	feature/PSO/出力数/viewport	固定三角形 Mesh Shader へ差替え
一部欠ける	bounds、index、出力上限	culling 全 off、meshlet ID 色
爆発する	stride、行列、局所 index	CPU 再構成と 1 meshlet 固定
ちらつく	race、barrier、LOD hysteresis	1 frame 固定、validation
GPU hang/reset	範囲外 write、巨大 dispatch	counts clamp、DRED/validation
遅い	pixel 律速、低 occupancy、帯域	timestamp 区間、simple PS

15.2 段階的な可視化 mode

最終 material だけでは原因を見分けにくい。次の debug mode を 1 キーで切り替える。

1. meshlet ID color
2. LOD level color
3. culling reason color
4. normal / tangent visualization
5. wireframe + bounds sphere/cone
6. primitive ID gradient

7. overdraw heatmap
8. quantization error amplification

ID color は hash から作るが、隣接 ID が似た色にならないよう bit mixing する。同じ meshlet ID が frame ごとに色を変えるなら、visible list または ID mapping が非決定的である。atomic compaction は順序が変わっても正しさに影響しない設計にする。

15.3 CPU reference を持つ

GPU だけでデータを検証しようとする、capture tool が読めない形式や race で詰まる。小モデルについて CPU で次を再現する。

- packed triangle の decode
- meshlet から global triangle list への再構成
- world/view/projection 変換
- sphere-frustum 判定
- LOD screen error
- normal cone 判定

CPU と GPU で同じ入力に対する可視 bit を buffer へ書き、差分 ID を表示する。floating-point 境界では epsilon の差があるため、完全一致を要求するケースと保守的包含を要求するケースを分ける。

15.4 canary と範囲検査

debug build では buffer 末尾に既知 pattern を置き、GPU 処理後に破壊されていないか確認する。shader 内で範囲外を検出したら、専用 error buffer へ最初の 1 件を atomic compare-exchange で記録する。

```

1 struct GpuError {
2     uint code;
3     uint meshletId;
4     uint value;
5     uint limit;
6 };
7 RWStructuredBuffer<GpuError> Error : register(u7);
8
9 void ReportError(uint code, uint meshletId,
10                 uint value, uint limit) {

```

```
11     uint original;  
12     InterlockedCompareExchange(Error[0].code,  
13                                 0, code, original);  
14     if (original == 0) {  
15         Error[0].meshletId = meshletId;  
16         Error[0].value = value;  
17         Error[0].limit = limit;  
18     }  
19 }
```

Listing 15.1: 最初の GPU validation error を記録する概念例

これは完全な race-free logging system ではないが、最初の壊れた meshlet へ到達する助けになる。error code を書いた lane 以外が field を書く競合を避けるには、lock value と memory ordering を含む設計へ発展させるか、固定 slot を meshlet ID で割り当てる。

15.5 GPU capture で見える順序

1. 問題 frame を capture する。
2. Mesh Dispatch call を選び、pipeline stage と shader binary を確認する。
3. bound descriptor/resource と offset/size を確認する。
4. indirect args の値を確認する。
5. meshlet descriptor、vertex refs、packed triangle を 1 個手計算する。
6. shader output/mesh viewer が使える tool なら、頂点と primitive を確認する。
7. Pixel Shader 入力、cull state、depth state へ進む。

capture 時だけ問題が消える場合、timing 依存 race、未初期化 memory、CPU/GPU lifetime を疑う。最適化を無効にした shader で消える場合も、compiler bug と断定する前に undefined behavior を探す。

15.6 timestamp query の設計

frame 全体だけでなく、次の区間を測る。

- cull compute または Task 付き geometry pass
- depth prepass
- main Mesh pass

- shadow pass
- Hi-Z build
- post process

query 結果の timestamp period を API property から変換し、queue 間の timestamp を直接比較しない。GPU clock calibration や CPU/GPU 相関が必要なら専用 API を使う。区間を細かくしすぎると query overhead と読みづらさが増えるため、仮説ごとに追加する。

15.7 代表的な profiler

表 15.2: 代表的な解析 tool と使い分け

tool	主な対象	本書で見るもの
RenderDoc	Vulkan / D3D12 capture	resource、pipeline、draw/dispatch、shader input/output
NVIDIA Nsight Graphics	NVIDIA GPU	GPU trace、shader/throughput、bottleneck
AMD Radeon GPU Profiler	Radeon GPU	wave、cache、pipeline busy、event timing
PIX on Windows	D3D12	timing capture、GPU capture、D3D12 validation
Vulkan validation	Vulkan correctness	API usage、同期、descriptor、best practices

tool の counter 名は architecture と version で変わる。「occupancy が低いから悪い」と単独で判断せず、実行時間、memory stall、ALU utilization、output/raster 量と合わせる。低 occupancy でも十分な ILP と cache hit で速い shader はある。

15.8 非決定性を制御する

atomic compaction の visible ID 順は実行ごとに変わり得る。opaque geometry で順序に依存しないなら正しい。しかし debug screenshot 比較や transparent sort では困る。

開発用に次を用意する。

- CPU culling mode
- GPU culling 後に ID sort する deterministic mode

- camera/time/random seed 固定
- 1 frame step
- asynchronous streaming 停止

正しさテストは deterministic mode、性能測定は本番 mode で行う。

演習 15.1: 欠ける geometry を切り分ける

「カメラを回すと一部 meshlet が消える」症状に対し、5 段階の切り分け手順を書け。視錐台、法線 cone、LOD、index、同期のどれかを特定できる順にすること。

演習 15.2: timestamp 区間を選ぶ

Task Shader 導入後に frame が 0.4ms 遅くなった。原因が Task 自体、Mesh 削減不足、Pixel 律速、Hi-Z 作成のどれかを調べるため、どの区間と counter を比較するか設計せよ。

既存レンダラから移行する

16.1 一度に置き換えない

既存レンダラへ Mesh Shader を追加するとき、asset、shader、pipeline、culling、LOD、tooling を同時に変更すると原因が追えない。次の段階で進める。

1. 固定 triangle の Mesh Pipeline を追加する。
2. 既存 vertex/index data を Mesh Shader がそのまま読む passthrough を作る。
3. offline meshlet 生成を追加し、見た目を従来経路と比較する。
4. Mesh Shader 内 0-output culling を追加する。
5. Task/Amplification compaction を追加する。
6. GPU-driven indirect と LOD を追加する。
7. 圧縮・animation・streaming を最適化する。

各段階で golden image、GPU time、memory、crash-free test を通す。前段階へ runtime で戻れるよう feature flag を残す。

16.2 共通 asset 表現

元の vertex/index buffer を捨てず、メッシュレット descriptor と局所 index を追加する。メッシュレットから従来 index list を作れるため、同じ頂点 order と material grouping を共有できる。

```
1 std::vector<std::uint32_t> buildLegacyIndices(  
2     std::span<const Meshlet> meshlets,  
3     std::span<const std::uint32_t> vertexRefs,  
4     std::span<const std::uint32_t> packedTriangles) {  
5  
6     std::vector<std::uint32_t> out;  
7     for (const Meshlet& m : meshlets) {  
8         for (std::uint32_t p = 0; p < m.primitiveCount; ++p) {  
9             const auto local = unpackTriangle(  

```

```

10         packedTriangles[m.primitiveOffset + p]);
11         out.push_back(vertexRefs[m.vertexOffset + local.x]);
12         out.push_back(vertexRefs[m.vertexOffset + local.y]);
13         out.push_back(vertexRefs[m.vertexOffset + local.z]);
14     }
15 }
16 return out;
17 }

```

Listing 16.1: メッシュレットから従来 index を再構成する

元 mesh の triangle 順を保持しない分割では、透明描画や triangle ID に依存する data が変わる。source primitive ID の mapping を保持するか、用途ごとに reorder 可否を決める。

16.3 shader code を共有する

position decode、normal decode、skinning、material evaluation を共通 function へ分ける。entry point だけ Vertex/Mesh で変える。

```

1 ProcessedVertex ProcessVertex(uint globalVertexId,
2                               uint instanceId) {
3     RawVertex raw = LoadVertex(globalVertexId);
4     ProcessedVertex v;
5     v.position = DecodePosition(raw);
6     v.normal   = DecodeNormal(raw);
7     if (IsSkinned(instanceId)) {
8         ApplySkinning(v, raw, instanceId);
9     }
10    return v;
11 }
12
13 // VSMain: からを得る SV_VertexIDglobalVertexId
14 // MSMMain: meshlet vertex からを得る referenceglobalVertexId

```

Listing 16.2: 共有 vertex 処理の骨格

API 差は macro で大量に隠すより、resource binding と entry interface を薄い adapter へ分ける。共通 function が GLSL/HLSL 両方で必要なら、共通中間言語、code generation、または HLSL 統一を検討する。

16.4 pipeline key と cache

Mesh Pipeline と Vertex Pipeline は別 PSO になる。既存 key に geometry path、meshlet profile、Task 有無を追加する。

$$K = (\text{material}, \text{pass}, \text{cull}, \text{depth}, \text{samples}, \text{geometryPath}, \text{profile}, \text{taskMode}). \quad (16.1)$$

key 爆発を防ぐため、dynamic state で扱えるもの、shader 内 branch で十分なもの、別 PSO が必要なものを分類する。起動時に全組み合わせを作らず、使用時に async compile し、fallback PSO を表示する仕組みも有効である。

16.5 テスト matrix

表 16.1: 最低限の cross-platform test matrix

OS/API	GPU	経路	検証
Windows/Vulkan	NVIDIA	Mesh+Task / Mesh / VS	image、validation、perf
Windows/Vulkan	Radeon	Mesh+Task / Mesh / VS	image、validation、perf
Linux/Vulkan	NVIDIA	Mesh+Task / Mesh / VS	X11/Wayland、driver 差
Linux/Vulkan	Radeon	Mesh+Task / Mesh / VS	RADV 等の実環境
Windows/D3D12	NVIDIA	Mesh+AS / Mesh / VS	debug layer、PIX
Windows/D3D12	Radeon	Mesh+AS / Mesh / VS	debug layer、RGP/PIX

Linux では使用 driver stack が異なるため、「Radeon なら同じ」とまとめない。起動ログに driver name/version と enabled extension を残す。ただし、対応可否は文字列ではなく feature query を真実とする。

16.6 失敗時の安全な降格

次の状態では Mesh 経路を無効化し、Vertex 経路へ戻す。

- feature または必要 property 不足
- shader/PSO 作成失敗
- asset profile が device 上限を超える
- known issue database で重大な driver 不具合に一致
- user 設定または remote kill switch

render 中の device loss 後に経路だけ切り替えて続行できるとは限らない。resource と device を再作成する通常の recovery 設計へ統合する。降格理由を UI と log に明示し、ユーザーに「Mesh Shader 対応 GPU なのに使われない」理由を説明できるようにする。

16.7 品質比較

従来経路と Mesh 経路を同じ frame で別 render target へ描き、差分を取る。

$$D(x, y) = |C_{mesh}(x, y) - C_{legacy}(x, y)|. \quad (16.2)$$

color 差だけでなく depth、normal、motion vector、object ID も比較する。primitive reorder により floating-point raster 差が出る可能性があるため、完全 bit 一致と許容誤差比較を分ける。欠落 pixel は小さな色差より重大として別 mask にする。

演習 16.1: 移行 milestone を定義する

既存 Vulkan renderer へ導入する 6 週間の milestone を作れ。各週に「機能」「テスト」「rollback 条件」を 1 つずつ含めること。

演習 16.2: pipeline key を削減する

material 20 種、pass 4 種、cull 2 種、MSAA 3 種、geometry path 3 種、profile 4 種、Task 2 種を全組み合わせにすると PSO はいくつか。実際には不要な軸の組み合わせを挙げ、削減案を示せ。

第 17

総合演習: メッシュレット・ビューアを作る

17.1 完成像

この章では、本書の内容を 1 つの小さな application へまとめる。最終成果物は、Windows/Linux で動く Vulkan 版を基準とし、Windows では D3D12 backend を追加できる構造にする。

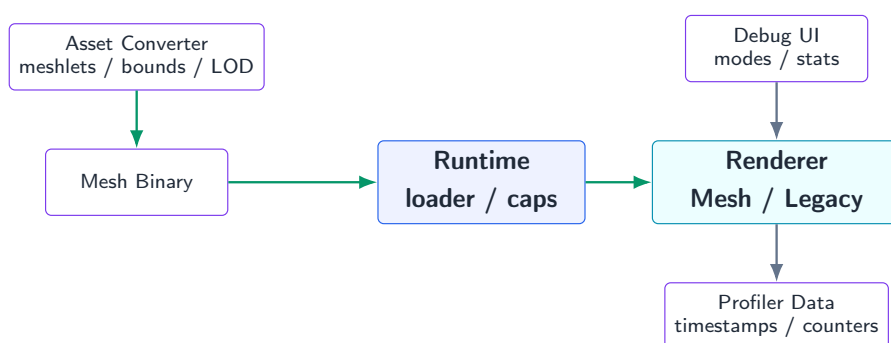


図 17.1: 総合演習の構成

17.2 directory 構成

```
1 meshlet-viewer/  
2   CMakeLists.txt  
3   assets/  
4   shaders/  
5     meshlet.mesh  
6     cull.task  
7     meshlet.frag  
8     legacy.vert  
9   src/  
10  app/  
11  asset/
```

```
12     graphics/common/  
13     graphics/vulkan/  
14     graphics/d3d12/           # optional  
15     tools/  
16     meshlet_converter/  
17     tests/  
18     asset_tests/  
19     image_tests/
```

backend 固有型を application 層へ漏らさず、DeviceCaps、MeshAsset、RenderStats を共通 interface にする。ただし薄い学習 project では過度な抽象化を避け、まず Vulkan を完成させてから共通部分を抽出する。

17.3 Milestone 0: baseline

通常 Vertex/Index pipeline で立方体と glTF model を描く。必要な機能は camera、depth buffer、resize、shader reload、GPU timestamp である。ここで golden image を作る。

チェックリスト

- Windows と Linux の両方で同じ model が出る。
- validation error が 0 件。
- front face、handedness、行列規約を README へ記す。
- GPU 名、driver、API version を log へ出す。

17.4 Milestone 1: feature detection と固定 triangle

第 5・6 章を実装する。Mesh Shader 非対応なら UI に理由を表示し、baseline を継続する。対応なら固定 triangle を別 mode で描く。

acceptance criteria:

- Mesh/Task feature と主要 property を画面表示。
- Mesh only で 3 頂点・1triangle を出力。
- capture 上で Vertex Shader が使われていない。
- cull mode 切替で巻き順を確認。

17.5 Milestone 2: converter と 1 meshlet

converter は入力 primitive を清掃し、最初は「全 triangle が上限内の小 model」を 1 meshlet へする。header、vertex refs、packed triangles を binary へ書く。runtime は 1 meshlet を読み、debug color で描く。

unit test:

1. pack/unpack が全 8bit 値の代表境界で一致。
2. binary offset/count の破損を拒否。
3. CPU 再構成 triangle が入力と一致。
4. GPU Mesh と legacy の depth 差が閾値以下。

17.6 Milestone 3: 複数 meshlet と bounds

貪欲分割、境界球、normal cone を追加する。UI で meshlet ID 色、sphere、cone axis を表示する。

表 17.1: 表示する asset 統計

項目	表示例
meshlet count	18,420
average vertices	58.3 / 64
average primitives	103.7 / 126
vertex utilization	91.1%
primitive utilization	82.3%
cone disabled	7.4%
asset bytes / triangle	18.6 B

17.7 Milestone 4: culling

次の順に追加し、各段階を toggle 可能にする。

1. object frustum culling
2. meshlet sphere frustum culling (Mesh 内 0 出力)
3. normal cone culling

4. Task/AS compaction

visible、frustum rejected、cone rejected、emitted mesh groups を GPU counter で数える。Task on/off の GPU 時間を同じ camera path で比較する。

17.8 Milestone 5: LOD

最初は model 全体の LOD を CPU で選び、screen error 式を検証する。次に cluster 単位へ進める。UI へ threshold slider、hysteresis、LOD color、triangle count を追加する。

LOD の acceptance criteria:

- camera を静止したとき LOD が往復しない。
- 親子の重複と穴がない。
- threshold を小さくすると単調に triangle 数が増える。
- 1080p と 4K で screen error の意味が一致する。

17.9 Milestone 6: indirect と同期

Compute culling から visible ID と indirect args を生成し、Mesh Dispatch へ渡す。同期 validation を有効にする。frame ring を 3 つ用意し、readback statistics は 2 から 3 frame 遅らせる。

stress test:

- camera teleport
- window resize 連打
- asset reload 連打
- 最大 meshlet 数 scene
- visible list capacity 直前と overflow
- Mesh/legacy 切替を毎 frame 行う debug mode

17.10 Milestone 7: cross-vendor tuning

P0 から P4 の複数 profile を生成し、NVIDIA/Radeon、Windows/Linux で同じ benchmark script を実行する。CSV へ次を保存する。

```
commit,gpu,driver,os,api,scene,profile,task,
```

```
2 resolution,visible_meshlets,emitted_prims,
3 gpu_cull_ms,gpu_mesh_ms,gpu_frame_ms,p95_ms
```

plot は GPU ごとに別 graph を作り、絶対時間と baseline 比の両方を示す。「勝った profile」だけでなく、scene 間のばらつきと worst case を報告する。

17.11 UI 設計

表 17.2: ビューアに必要な操作

panel	control / 表示
Device	GPU、driver、API、Mesh/Task support、limits
Pipeline	Mesh+Task / Mesh only / Legacy、wireframe、cull mode
Culling	frustum、cone、occlusion、freeze culling camera
LOD	threshold、hysteresis、level color、freeze LOD
Meshlets	ID color、bounds、cone、selected meshlet data
Performance	GPU 区間、counts、profile、CSV capture
Validation	first GPU error、overflow、asset warnings

UI 自体が GPU 時間へ影響するため、benchmark mode では非表示にし、camera path を script 再生する。

17.12 提出物と評価 rubric

表 17.3: 総合演習の自己評価 rubric

項目	配点	完成条件
正しさ	25	validation 0、legacy 差分、bounds/index 検証
機能検出/fallback	10	非対応でも起動、理由表示
meshlet converter	15	再現可能、統計、binary version
culling/LOD	15	toggle、counter、保守的判定
同期/lifetime	10	stress test、ring、overflow 安全
cross-vendor	10	NVIDIA/Radeon、Win/Linux 結果
性能分析	10	timestamp、仮説、比較表
説明/README	5	build、規約、制約、再現手順

演習 17.1: 総合設計書を作る

ビューアの resource 一覧、pass graph、buffer layout、feature fallback、frame ring、debug modes を A4 2 ページ以内の設計書にまとめよ。各 resource について producer と consumer を書くこと。

演習 17.2: 性能結論を書く

2 つの GPU、3 つの scene、3 つの meshlet profile、Task on/off を測ったと仮定し、結論 template を作れ。「最速値」「安定 default」「例外 scene」「未検証事項」「次の実験」を必ず含めること。

17.13 ここまでで身に付いたこと

メッシュシェーダーを使うとは、単に Vertex Shader を別の stage へ書き換えることではない。geometry をメッシュレットへ再編し、GPU 実行単位、bounds、LOD、buffer layout、同期、fallback、計測を一つの system として設計することである。

初心者が最初に守るべき原則を 5 つにまとめる。

要点

1. 固定 triangle から始め、変更を 1 つずつ増やす。
2. 上限は仕様の数字を決め打ちせず、実機 property を読む。
3. NVIDIA/Radeon を別コードにせず、共通 default と実測 profile を持つ。
4. culling は保守的にし、破棄理由を可視化する。
5. performance claim には camera path、timestamp、counter、fallback 比較を添える。

数学・GPU 演算早見表

A.1 ベクトル

表 A.1: メッシュシェーダーで使うベクトル演算

目的	式	注意
長さ	$\ \mathbf{v}\ = \sqrt{\mathbf{v} \cdot \mathbf{v}}$	比較だけなら平方根を避け、長さの二乗を使う
正規化	$\hat{\mathbf{v}} = \mathbf{v} / \ \mathbf{v}\ $	0 付近を guard する
内積	$\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$	単位 vector 同士なら角度の cos
外積	$\mathbf{a} \times \mathbf{b}$	頂点巻き順で符号が反転
距離	$d = \ \mathbf{p} - \mathbf{q}\ $	最近傍比較は d^2 でよい
線形補間	$\text{lerp}(a, b, t) = a(1 - t) + bt$	t 範囲を意図的に clamp するか決める
成分積	$\mathbf{a} \odot \mathbf{b}$	scale や量子化復元で使用

A.2 変換

列ベクトル規約の代表式を示す。

$$\mathbf{p}_{world} = M\mathbf{p}_{object}, \quad (\text{A.1})$$

$$\mathbf{p}_{view} = V\mathbf{p}_{world}, \quad (\text{A.2})$$

$$\mathbf{p}_{clip} = P\mathbf{p}_{view}, \quad (\text{A.3})$$

$$\mathbf{p}_{ndc} = (x_c/w_c, y_c/w_c, z_c/w_c). \quad (\text{A.4})$$

法線は位置と異なり、translation を受けない。非一様 scale がある一般形は

$$\mathbf{n}_{world} = \text{normalize} \left((M_{3 \times 3}^{-1})^T \mathbf{n}_{object} \right). \quad (\text{A.5})$$

負 scale では向きと front face の関係が反転する可能性がある。model matrix determinant の符号を instance flag へ持たせる設計もある。

A.3 カリング

表 A.2: 代表的なカリング判定

判定	条件	備考
球 vs 平面	$\mathbf{n} \cdot \mathbf{c} + d < -r$ なら外	$\mathbf{n}$ を単位長にする
球 vs 視錐台	6 平面のどれかで外なら reject	内向き法線規約の例
画面誤差	$e_{px} \approx eH/(2z \tan(f_y/2))$	near/背後を先に処理
画面半径	$r_{px} \approx rH/(2z \tan(f_y/2))$	small object 判定
法線 cone	axis と view 方向の内積を cutoff 比較	cutoff 定義と球の角度を考慮
AABB vs plane	center-extents 法で投影半径を比較	非一様 scale 後も扱いやすい

AABB の center を $\mathbf{c}$ 、half extents を $\mathbf{e}$ とすると、平面法線への投影半径は

$$r_p = |n_x|e_x + |n_y|e_y + |n_z|e_z. \quad (\text{A.6})$$

$\mathbf{n} \cdot \mathbf{c} + d < -r_p$ なら完全に外である。

A.4 並列 compaction

可視 flag $b_i \in \{0, 1\}$ に対し、exclusive prefix sum

$$s_i = \sum_{j=0}^{i-1} b_j \quad (\text{A.7})$$

を求める。 $b_i = 1$ の lane が $\text{output}[s_i] \leftarrow \text{ID}$ を書く。総可視数は $\sum_i b_i$ である。subgroup ballot が使える場合、bit count で prefix を得られる。ワークグループが複数 subgroup を含む場合は、subgroup ごとの count をさらに scan する。

A.5 容量計算

$$\text{vertex output bytes} \approx V \times \text{vertex attribute stride}, \quad (\text{A.8})$$

$$\text{primitive output bytes} \approx P \times \text{primitive attribute stride}, \quad (\text{A.9})$$

$$\text{triangle local index bytes} = P \times 3 \times \text{index byte width}. \quad (\text{A.10})$$

実際の hardware 内部形式は API 上の型と同じとは限らない。容量計算は上限検査と相対比較に使い、occupancy は profiler で確認する。

Vulkan と Direct3D 12 対応表

B.1 stage・command・intrinsic

表 B.1: Mesh Shader API 対応表

概念	Vulkan / GLSL	D3D12 / HLSL
機能単位	VK_EXT_mesh_shader	Mesh Shader Tier 1
前段 stage	VK_SHADER_STAGE_TASK_BIT_EXT	Amplification Shader / as_6_5
本体 stage	VK_SHADER_STAGE_MESH_BIT_EXT	Mesh Shader / ms_6_5
直接起動	vkCmdDrawMeshTasksEXT	DispatchMesh API
間接起動	vkCmdDrawMeshTasksIndirectEXT	ExecuteIndirect
count 付き間接	vkCmdDrawMeshTasksIndirectCountEXT	count buffer 付き ExecuteIndirect
出力数	SetMeshOutputsEXT	SetMeshOutputCounts
子 Mesh 発行	EmitMeshTasksEXT	AS 内 DispatchMesh
頂点位置	gl_MeshVerticesEXT[i].gl_Position	vertex output の SV_Position
triangle index	gl_PrimitiveTriangleIndicesEXT	out indices uint3[]
per primitive	perprimitiveEXT	out primitives
primitive cull	gl_CullPrimitiveEXT	SV_CullPrimitive
group 内番号	gl_LocalInvocationIndex	SV_GroupIndex
group 番号	gl_WorkGroupID	SV_GroupID
Task payload	taskPayloadSharedEXT	groupshared payload

B.2 Vulkan の機能構造

表 B.2: Vulkan で確認する主な構造と field

構造	主な用途
VkPhysicalDeviceMeshShaderFeaturesEXT	taskShader, meshShader, queries、multiview 関連
VkPhysicalDeviceMeshShaderPropertiesEXT	workgroup、output、shared memory、payload、preference
VkDrawMeshTasksIndirectCommandEXT	X/Y/Z group count の indirect argument
VkPipelineShaderStageCreateInfo	Task/Mesh module と entry point
VkBufferMemoryBarrier2	Compute/Transfer から Task/Mesh/Indirect への同期

properties の field は SDK header と最新 reference page で確認する。app が必要とする最低値を明文化し、asset profile を選ぶ。

```

1 struct MeshShaderCaps {
2     bool meshShader = false;
3     bool taskShader = false;
4     std::uint32_t maxMeshInvocations = 0;
5     std::uint32_t maxOutputVertices = 0;
6     std::uint32_t maxOutputPrimitives = 0;
7     std::uint32_t maxTaskPayloadBytes = 0;
8     std::uint32_t subgroupSize = 0;
9 };

```

Listing B.1: device capability を正規化する構造

Vulkan と D3D12 の問い合わせ結果をこの共通構造へ変換し、上位 renderer は profile selection だけを見る。API 固有の追加 field は backend 側へ保持する。

B.3 Direct3D 12 の必須規模

Microsoft 仕様が示す Mesh Shader の必須 support には、最大 128 threads、最大 256 output vertices、最大 256 output primitives、最大 16KiB payload、最大 32KiB Mesh output data などがある [1]。実際の PSO 作成と shader validator は、宣言が device/runtime 制約を満たすか検査する。

注意

「D3D12 で 256 まで必須」だから Vulkan でも 256 を無条件に使う、という推論はしない。Vulkan は `VkPhysicalDeviceMeshShaderPropertiesEXT` の値を読む。共通 asset profile は、対象 device 集合の最小値を満たすか、複数 profile を用意する。

B.4 shader interface の確認事項

チェックリスト

- 出力 topology が triangle/line/point の意図と一致する。
- runtime 出力数が compile-time 最大値以下。
- 全使用頂点と全 primitive index を 1 回以上書く。
- primitive index の各成分が出力頂点数未満。
- Mesh と Fragment/Pixel の location または semantic が一致する。
- per-primitive 属性の qualifier/semantic が一致する。
- Task/AS payload 型と Mesh 入力型の size/layout が一致する。
- workgroup threads が device 上限以下。

B.5 feature fallback 表

表 B.3: 機能組み合わせごとの経路

Mesh	Task/AS	選択
✗	—	Vertex/Index fallback
✓	✗	Mesh only。Mesh 内 0-output culling または Compute list
✓	✓	Mesh+Task/AS。採算を測り runtime で on/off

演習問題の解説

解説: 演習 1.1

ワークグループ: 複数の shader invocation が共有 memory と barrier を使って 1 つの小さな仕事を協調処理する単位。**メッシュレット:** 頂点数と triangle 数を小さな上限へ収めた局所 mesh で、1 Mesh Shader group の入力単位。**Task/Amplification Shader:** 候補を判定・圧縮し、必要な Mesh Shader group 数と payload を発行する任意の前段。Mesh Shader は API から直接起動できるため、前段なしでも頂点と primitive を出力できる。

解説: 演習 1.2

(b) 都市と (d) 大量の岩が有力である。細粒度 frustum/occlusion、LOD、GPU-driven dispatch の削減余地が大きい。(a) は 1 triangle しかなく、(c) も geometry 量が小さいため、通常 path の単純さが勝ちやすい。

解説: 演習 2.1

頂点は for ($v=\text{lane}$; $v<64$; $v+=32$)、triangle は for ($p=\text{lane}$; $p<126$; $p+=32$) とする。頂点は lane 0 から 31 が各 2 個。triangle は lane 0 から 29 が 4 個、lane 30 と 31 が 3 個を担当する。最大担当数は頂点 2、triangle 4 である。

解説: 演習 2.2

正しさだけなら、実際の count が 40 以下で全 index が範囲内なら描ける。性能面では、compile-time 最大出力が大きいため内部 output storage や occupancy を圧迫し得る。64 など実データに近い variant を使い、device profiler で比較する。

解説: 演習 3.1

頂点参照表は初出順に [42, 18, 97, 101, 56]。局所 triangle は (0, 1, 2), (2, 1, 3), (2, 3, 4) となる。

解説: 演習 3.2

頂点参照は $64 \times 4 = 256\text{B}$ 。(a) triangle は $126 \times 3 \times 4 = 1512\text{B}$ 、合計 1768B 、header 込み 1784B 。(b) triangle は $126 \times 3 = 378\text{B}$ 、合計 634B 、header 込み 650B 。32bit packed element へ 1triangle ずつ格納する実装なら triangle は 504B となり、header 込み 776B である。

解説: 演習 4.1

中心 $(-3.2, 0, 0)$ では signed distance は $-3.2 + 2 = -1.2$ 。 $-1.2 < -0.5$ なので完全に外。中心 $(-2.3, 0, 0)$ では -0.3 で、 $-0.3 < -0.5$ は偽。平面と交差または内側なので reject しない。

解説: 演習 4.2

$s = 1080 / (2 \tan 30^\circ) \approx 935.3$ 。 $e_{px} = 0.02 \times 935.3 / 10 \approx 1.87\text{px}$ 。距離が 5m なら約 3.74px で 2 倍になる。

解説: 演習 4.3

表には入力 position、model matrix、期待 screen 位置、期待 front/back、期待 normal を置く。X 回転で up 軸、負 scale で巻き順、非一様 scale で normal inverse-transpose を検査できる。同じ固定 case を CPU math test と screenshot test へ使うと、転置や API 設定の変更を早期検出できる。

解説: 演習 5.1

例: GPU 名、vendor/device ID、driver、OS/API、Mesh extension/Tier、Mesh feature、Task feature、max workgroup invocations、max output vertices、max output primitives、shared memory、payload、subgroup size、選択 profile、選択 path。上限値と選択結果を同時に出すことが重要である。

解説: 演習 5.2

shader/PSO/draw command は Mesh と Legacy を二重化する。頂点属性、material、instance data は共有する。asset は meshlet data と再構成可能な legacy index を持つ。統計は共通名へ正規化し、Mesh 固有 counter は 0 または N/A とする。

解説: 演習 6.1

4 頂点を四角の角へ置き、triangle を $(0, 1, 2)$, $(0, 2, 3)$ とする。SetMeshOutputsEXT(4,2) または SetMeshOutputCounts(4,2) を呼ぶ。2 枚とも同じ向きから見て反時計回りなど、front face 規約を揃える。

解説: 演習 6.2

lane 0,1,2 の色を RGB へ割り当てる。Fragment/Pixel Shader 入力は triangle 頂点間で barycentric 補間されるため、内部 pixel は 3 色の重み付き和になる。flat/nointerpolation にすると補間されない。

解説: 演習 7.1

CPU では、少なくとも次を検証する。

- 頂点数・primitive 数が profile 上限以内か。
- offset と count の加算が各 section 範囲内か。
- 局所 index が meshlet 頂点数未満か。
- global index が position 配列の要素数未満か。

error には asset 名、LOD、meshlet ID、primitive/local vertex、実値、上限、file offset を含める。

解説: 演習 7.2

$R = 378/64 \approx 5.91$ 。126 頂点なら $R = 3.0$ 。前者は局所頂点再利用が高い。ただし global memory locality、出力上限/occupancy、bounds の大きさ、attribute decode cost、culling 可視率などはこの指標だけでは分からない。

解説: 演習 8.1

exclusive prefix は $[0, 1, 1, 2, 3, 3, 3, 4]$ 。可視 lane 0,2,3,6 が slot 0,1,2,3 へ書く。visible count は 4 である。

解説: 演習 8.2

初期化後 barrier を外すと、一部 lane が古い visibleCount へ atomic add する可能性がある。atomic 後 barrier を外すと、Emit/Dispatch が count や payload 書き込み完了前に値を読む可能性がある。GPU lane の相対実行順は未定義で、source code の行順は lane 間同期を意味し

ない。

解説: 演習 9.1

1080p では 0.8px なので閾値 1.0px 以下、粗い LOD を選べる。4K では 1.6px なので子へ降り、細かい LOD を選ぶ。距離だけでは解像度による pixel 密度の差を含められない。

解説: 演習 9.2

擬似コードは「外なら return」「error が閾値以下または leaf なら親/現在 node を append して return」「それ以外は子だけを再帰評価」とする。append 後に子へ進まず、子へ進む branch では親を append しないため排他的になる。

解説: 演習 10.1

(a) A は $1 + 0.4 = 1.4$ 、B は $2 + 0.1 = 2.1$ なので A。 (b) A は $1 + 5 \times 0.4 = 3.0$ 、B は $2 + 5 \times 0.1 = 2.5$ なので B。頂点再利用重視では A、空間的に小さな bounds 重視では B が選ばれる。

解説: 演習 10.2

まず $\text{elementSize} \neq 0$ 、 $\text{count} \leq \text{MAX}/\text{elementSize}$ を確認して $\text{bytes} = \text{count} * \text{elementSize}$ を得る。次に $\text{offset} \leq \text{fileSize}$ 、 $\text{bytes} \leq \text{fileSize} - \text{offset}$ を順に確認する。section 同士の重複が許されない format なら、全 range を sort して交差も検査する。

解説: 演習 11.1

100m では $100/65535 \approx 0.001526\text{m}$ 、約 1.526mm/step。1m cluster では約 0.00001526m、0.01526mm、約 15.3 μm /step。局所 AABB の方が約 100 倍細かい。

解説: 演習 11.2

Depth-only は position、shadow は position と skin、G-buffer は position/normal/tangent/UV/material、transparent はそれらに color 等を使う。position を独立 stream、normal+tangent、UV+color、skin joints+weights を分ける案なら、pass ごとに不要 stream を読まずに済む。

解説: 演習 12.1

頂点上限は $256/4 = 64$ 粒子、primitive 上限は $256/2 = 128$ 粒子なので、出力数の制約は 64 粒子。1 頂点 32B なら 64 粒子の頂点属性は $64 \times 4 \times 32 = 8192\text{B}$ で 32KiB 未満。primitive 属性や alignment を加えても、この条件では頂点数上限が先に効く。

解説: 演習 12.2

画面外 70% なら object-level bounds で先に character を除外し、残り 30 体を Compute skin して 3 pass で再利用する案が有力。meshlet bounds は clip ごとの保守 bounds でまず cull し、必要なら Compute skin 中に更新する。Mesh 内 skin は pass ごとに重複するため、3 pass では不利になりやすいが、小さな可視部分だけを処理できる scene では比較対象に残す。

解説: 演習 13.1

共通 src は Compute Shader stage / shader storage write。A の dst は Mesh Shader stage / shader storage read。B の dst は Draw Indirect stage / indirect command read。buffer ごとに barrier を分けるか、同一 buffer なら stage/access を OR し、正しい range を指定する。

解説: 演習 13.2

visible ID は $200000 \times 4 = 800000\text{B}$ で、ちょうど 256B の倍数。args 16B を置き、次の statistics を 256B 境界へ合わせると frame end は 800512B。3 frame で 2401536B、約 2.29MiB である。header や余裕分、overflow counter を加える場合はさらに round up する。

解説: 演習 14.1

threads 32 では $E_v = 50/64 = 78.125\%$ 、 $E_p = 100/128 = 78.125\%$ 。threads 64 でも $E_v = 50/64$ 、 $E_p = 100/128$ で同じ。実速度は subgroup 構成、register/shared memory による occupancy、memory latency、loop overhead、compiler scheduling などで変わる。

解説: 演習 14.2

列の例: GPU/driver、OS/API、scene、profile、Task、threads、meshlet count、visible ratio、vertex/primitive utilization、emitted primitives、Task ms、Mesh ms、frame ms、p95、VRAM、shader hash。各 GPU で同じ camera path と build を使い、baseline 比も記録する。

解説: 演習 15.1

(1) culling を全 off にし、残るなら index/data へ進む。(2) frustum だけ on、sphere を表示して平面符号を確認。(3) cone だけ追加し、両面 flag と axis/cutoff を確認。(4) LOD を固定し、親子重複/穴を確認。(5) 1 meshlet 固定と CPU 再構成で index を確認し、最後に同期 validation と frame freeze で race を調べる。

解説: 演習 15.2

Task/AS 区間、Mesh+raster 区間、Pixel 区間、Hi-Z build 区間を別 timestamp にする。候補/可視/発行 group 数、emitted primitives、pixel samples、Task occupancy/memory stall を比較する。Task 時間が +0.4ms で Mesh 削減が小さければ採算不足、Mesh は減るが frame が同じなら Pixel 律速、Hi-Z 追加分なら culling pipeline 全体の損益を見る。

解説: 演習 16.1

例: 第 1 週は固定 triangle と feature query、第 2 週は passthrough Mesh、第 3 週は converter、第 4 週は culling、第 5 週は Task/indirect、第 6 週は cross-platform tuning。各週で legacy screenshot との差、validation、性能を test し、crash、欠落、目標超過なら前週 feature flag へ rollback する。

解説: 演習 16.2

全組み合わせは $20 \times 4 \times 2 \times 3 \times 3 \times 4 \times 2 = 11520$ PSO。Legacy では meshlet profile/Task 軸が不要、Mesh only では Task 軸が固定、shadow/depth では material variant を統合できる場合がある。MSAA や cull を dynamic state へ移す、使用 profile を asset ごとに 1 つへ絞る、on-demand compile で大幅に減らせる。

解説: 演習 17.1

設計書の最低構成は、pass graph、resource table、buffer byte layout、feature decision tree、3-frame ring 図、debug mode 一覧である。resource table には name、size、lifetime、producer stage/access、consumer stage/access、fallback 利用を記す。

解説: 演習 17.2

結論例: 「P2+Task が都市 scene の NVIDIA で最速、P1 Mesh-only が全 GPU/scene で worst case の小さい default。草原では Task overhead が上回る。Radeon Linux の occlusion path

は未検証。次は Task 候補数 32/64 と payload 128/256B を比較する。」絶対 ms、baseline 比、p95、測定条件を併記する。

用語集

表 D.1: メッシュシェーダー関連用語

用語	説明
Amplification Shader	D3D12 で Mesh Shader group 数と payload を発行する前段。Vulkan の Task Shader に相当。
AABB	Axis-Aligned Bounding Box。座標軸に平行な境界箱。
AoS	Array of Structures。頂点ごとに属性をまとめる配置。
back-face culling	camera から背を向けた triangle を rasterize しない処理。
barrier	producer の実行・memory write を consumer から利用可能にする同期命令。
bindless	多数 resource を descriptor index で選ぶ設計の総称。対応 feature と範囲検査が必要。
compaction	flag が真の要素だけを連続 output へ詰める並列処理。
cone culling	meshlet 内の normal 範囲を cone で表し、全体が背面なら捨てる処理。
clip space	projection 後、perspective divide 前の 4 次元座標空間。
DRED	Device Removed Extended Data。D3D12 device removal 解析の仕組み。
DXC	DirectX Shader Compiler。HLSL を DXIL または SPIR-V へ compile する。
DXIL	DirectX Intermediate Language。D3D12 shader の中間表現。
frustum	camera が見る錐台。通常 left/right/top/bottom/near/far の 6 平面で表す。
GPU-driven	CPU が個別 draw を列挙せず、GPU が可視 list や indirect argument を生成する設計。
groupshared/shared memory	1 workgroup 内 invocation が共有する高速 memory。barrier が必要。

用語	説明
Hi-Z / depth pyramid	depth image を階層化し、occlusion test へ使う。
hysteresis	状態を切り替える往路と復路の閾値を分け、ちらつきを抑える。
indirect dispatch	group 数などの argument を GPU buffer から読む起動方式。
invocation	shader entry point の 1 実行。thread/lane という語と文脈上近い。
LOD	Level of Detail。距離や画面誤差に応じて geometry 詳細度を変える。
meshlet	小さな頂点・primitive 集合。1 Mesh Shader workgroup の典型入力単位。
Mesh Shader	workgroup で協調し、頂点と primitive を直接出力する shader stage。
NDC	Normalized Device Coordinates。perspective divide 後の正規化座標。
normal cone	meshlet 内面 normal を包含する axis と半角/cutoff。
occlusion culling	他の geometry に完全に隠れた bounds を描かない処理。
octahedral encoding	単位 normal を正八面体写像で 2 成分へ圧縮する方法。
occupancy	hardware 上で同時常駐できる wave/warp/workgroup の度合い。高ければ常に速いとは限らない。
payload	Task/Amplification Shader から子 Mesh Shader groups へ渡す共有 data。
per-primitive attribute	頂点ではなく triangle/line/point 単位で持つ属性。
prefix sum / scan	各要素より前の合計を並列計算し、compaction slot を求める演算。
primitive	rasterizer へ渡す点、線、triangle。本書では主に triangle。
PSO	Pipeline State Object。shader と固定 function state の組み合わせ。
quantization	continuous 値を有限 bit の整数表現へ写す。誤差評価が必要。
Radeon	AMD の GPU 製品系列。本書では API feature query を優先し、名前決め打ちを避ける。
RGP	Radeon GPU Profiler。AMD GPU の event timing や hardware counter 解析 tool。
SoA	Structure of Arrays。属性ごとに連続配列へ分ける配置。
SPIR-V	Vulkan などで行われる shader 中間表現。
subgroup	hardware が近接して実行する invocation 集合。NVIDIA warp、AMD wave に対応する概念。

用語	説明
Task Shader	Vulkan で候補を判定し、Mesh Shader group 数と payload を発行する前段。
UAV	D3D の unordered access resource view。shader から read/write できる。
validation layer	Vulkan API 利用や同期の誤りを検査する layer。
vertex reuse	複数 triangle が同じ局所頂点を共有し、decode/read を再利用する度合い。
wave / warp	AMD / NVIDIA で使われる SIMD 実行 group の呼称。size を決め打ちしない。
workgroup	shared memory と barrier を共有し、1 つの仕事を協調する invocation 集合。

参考資料

以下は本書の仕様確認と追加学習に使った公式資料である。API と driver は更新されるため、実装時は各資料の最新版を確認する。参照日は 2026 年 8 月 24 日。

参考文献

- [1] Microsoft, “Mesh Shader”, DirectX-Specs.
<https://microsoft.github.io/DirectX-Specs/d3d/MeshShader.html>
- [2] Microsoft, “HLSL Shader Model 6.5”, DirectX-Specs.
https://microsoft.github.io/DirectX-Specs/d3d/HLSL_ShaderModel6_5.html
- [3] Microsoft Learn, “D3D12_MESH_SHADER_TIER enumeration”.
https://learn.microsoft.com/en-us/windows/win32/api/d3d12/ne-d3d12-d3d12_mesh_shader_tier
- [4] Khronos Group, “VK_EXT_mesh_shader”, Vulkan Reference Pages.
https://docs.vulkan.org/refpages/latest/refpages/source/VK_EXT_mesh_shader.html
- [5] Khronos Group, “GLSL_EXT_mesh_shader”.
https://github.com/KhronosGroup/GLSL/blob/main/extensions/ext/GLSL_EXT_mesh_shader.txt
- [6] Khronos Group, “SPV_EXT_mesh_shader”.
https://github.com/khronos.org/SPIRV-Registry/extensions/EXT/SPV_EXT_mesh_shader.html
- [7] Khronos Group, “HLSL in Vulkan”.
<https://docs.vulkan.org/guide/latest/hlsl.html>
- [8] Microsoft, “DirectXShaderCompiler SPIR-V CodeGen”.
<https://github.com/microsoft/DirectXShaderCompiler/blob/main/docs/SPIR-V.rst>
- [9] Khronos Group, “Mesh shading”, Vulkan Samples.
https://docs.vulkan.org/samples/latest/samples/extensions/mesh_shading/README.html
- [10] Khronos Group, “Mesh shader culling”, Vulkan Samples.
https://docs.vulkan.org/samples/latest/samples/extensions/mesh_shader_culling/README.html
- [11] Khronos Group, “From geometry shader to mesh shader”, Vulkan Samples.
https://docs.vulkan.org/samples/latest/samples/extensions/gshader_to_mshader/README.html
- [12] Khronos Group, “Vulkan Samples Build Guide”.

-
- <https://github.com/KhronosGroup/Vulkan-Samples/blob/main/docs/build.adoc>
- [13] Microsoft, “DirectX-Graphics-Samples”.
<https://github.com/microsoft/DirectX-Graphics-Samples>
- [14] AMD GPUOpen, “Mesh Shaders introduction”.
https://gpuopen.com/learn/mesh_shaders/mesh_shaders-index/
- [15] AMD GPUOpen, “Mesh shaders optimization and best practices”.
https://gpuopen.com/learn/mesh_shaders/mesh_shaders-optimization_and_best_practices/
- [16] NVIDIA Developer Blog, “Introduction to Turing Mesh Shaders”.
<https://developer.nvidia.com/blog/introduction-turing-mesh-shaders/>
- [17] NVIDIA Developer Blog, “Advanced API Performance: Mesh Shaders”.
<https://developer.nvidia.com/blog/advanced-api-performance-mesh-shaders/>
- [18] NVIDIA Developer Blog, “Using Mesh Shaders for Professional Graphics”.
<https://developer.nvidia.com/blog/using-mesh-shaders-for-professional-graphics/>
- [19] AMD GPUOpen, “AMD RDNA 2 Architecture”.
<https://gpuopen.com/rdna2/>
- [20] Mesa Project, “Mesa 23.0.0 Release Notes”.
<https://docs.mesa3d.org/relnotes/23.0.0.html>

あとがき

メッシュシェーダーは、GPU programming の複数分野を一度に学べる題材である。小さな triangle を出すだけなら短い shader で済むが、真価を引き出すには meshlet 生成、数学、memory layout、parallel compaction、synchronization、profiling、fallback を結び付ける必要がある。

最初の目標は最速の renderer ではない。固定 triangle、1 meshlet、複数 meshlet、frustum、Task、LOD という順序を守り、各段階で「何を入力し、何を出力し、どの上限と同期に依存するか」を説明できる renderer を作ることである。その土台があれば、新しい GPU や driver、API revision に対しても、名前ではなく feature と計測で適応できる。

まずは1枚の三角形から。

そして、1 つずつ測りながら育てよう。